

.v file

Tuesday, September 08, 2026 11:42 AM

1. WHAT IS A .v FILE? WHY IS IT REQUIRED? WHERE DOES IT COME FROM? WHO CREATES IT?

Complete understanding for ASIC Design & Physical Design perspective

1 What is a .v file?

- A .v file is a **text-based hardware description language (HDL) file** written in Verilog.
- It describes the digital logic of a design in a human-readable form using constructs like module, input, output, wire, reg, always, assign, case, if-else, etc.
- It captures the **functionality** (what the design should do), not the physical implementation (how it is built).
- EDA tools read the .v file and convert it into **actual gates and interconnections**.

.v file = Text Description of Hardware

2 Why is a .v file required in an ASIC flow?

- Allows designers to **describe complex hardware** in text instead of drawing millions of gates manually.
- Acts as the **main input to EDA tools** (DC/Genus) for synthesis.
- Enables functional simulation, verification, synthesis, DFT, and physical design.
- Helps in **design reuse, parameterization**, and easy modification.
- Used throughout the complete ASIC flow: RTL → Simulation → Synthesis → PnR → Signoff.
- Without a .v file, tools have no information about the design logic.

The .v file is the bridge between the designer's idea and the final silicon.

3 Where does a .v file come from?

- The initial .v file (RTL) is created by the RTL / Front-End design team based on the design specification.
- It is written to meet functional requirements and architecture.
- After RTL verification, the .v file is given to synthesis tools (Synopsys DC / Cadence Genus) along with technology libraries and constraints (SDC).
- Synthesis converts the RTL .v file into a new .v file – gate-level netlist using real standard cells from the technology library.
- This synthesized .v file is then used for DFT, physical design (Innovus), STA, and signoff.

4 Who creates the .v file?

RTL .v file

Created by RTL / Front-End Designers (hardware design team)

Based on specification (architecture + functional requirements)

↓

Gate-level netlist .v file

Generated automatically by synthesis tools (Synopsys DC / Cadence Genus)

Using technology libraries (.lib) and constraints (SDC)

5 .v file in ASIC Design Flow

The flowchart shows the progression from Specification (Functional Requirements) by the System / Product Team, through RTL Design (.v file) by the RTL / Front-End Designer, Verification (Simulation) by the Verification Team (Modelsim, VCS, etc.), Synthesis (DC / Genus) which generates Gate-level .v (netlist), Physical Design (Innovus) which uses netlist .v for PnR (Floorplan, Placement, CTS, Routing), and finally Signoff (STA, DRC, LVS) resulting in the Final GDSII (Tapeout).

6 Example: RTL .v file (What designer writes)

```

mimo_rx_chain.v
RTL (Behavioral)

module mimo_rx_chain (
    input wire clk_rf,
    input wire rst_n_rf,
    input wire [15:0] adc_data_i,
    output reg [31:0] rx_data_out,
    output wire rx_overflow
);

// Simple example logic
always @(posedge clk_rf or negedge rst_n_rf) begin
    if (!rst_n_rf)
        rx_data_out <= 32'd0;
    else
        rx_data_out <= adc_data_i;
end
endmodule

```

Describes what the circuit should do (Functional/Behavioral level)

7 Example: Gate-level .v file (What synthesis generates)

```

mimo_rx_chain.v
Gate-Level (Structural)

module mimo_rx_chain (
    input clk_rf,
    input rst_n_rf,
    input [15:0] adc_data_i,
    output [31:0] rx_data_out,
    output rx_overflow
);

DFF_X1 U1 ( .D(n1), .CLK(clk_rf), .Q(rx_data_out[0]),
    .RN(rst_n_rf) );
AND2_X1 U2 ( .A(adc_data_i[0]), .B(n2), .ZN(n1) );
INV_X1 U3 ( .A(rx_data_out[0]), .ZN(n2) );
...
endmodule

```

Describes how the circuit is actually built (Structural level with real standard cells)

8 Key Comparison

Feature	RTL .v	Gate-level .v
Created by	RTL designer	Synthesis tool
Describes	What it should do (behavior)	How it is built (structure)
Level	Behavioral	Structural
Contains	always, assign, if-else, case	Standard cell instances + nets
Has timing?	No (just intent)	Yes (from .lib)
Example	add a and b	AND2_X1 U1 ...
Used by	Simulation, Synthesis	Physical Design, STA, DFT, Signoff

9 Key Takeaways

- .v file is a Verilog HDL file
- Text-based description of hardware
- Required for entire ASIC flow
- Two main forms: RTL and gate-level
- RTL .v written by designers
- Gate-level .v generated by synthesis tools
- Same .v extension, different content and purpose
- Essential for PD engineers to understand structure, hierarchy, ports, and logic when debugging issues

10 Common Interview Questions

- What is a .v file?
- Why is a .v file required in an ASIC flow?
- Who creates the .v file?
- Where does a .v file come from?
- What are the types of .v files?
- Can we use RTL .v directly in PnR?

Remember: A .v file is the starting point of the digital design. It captures the designer's intent and is the bridge between the idea and the final silicon. "Text to Gates, Then to GDSII"

2. WHAT DOES A .v FILE CONTAIN? HOW IS IT USED FROM RTL TO GDSII? DIFFERENCE BETWEEN VERILOG FILE AND NETLIST? WHY SHOULD A PD ENGINEER UNDERSTAND A .v FILE?

From design intent to silicon — understanding .v file content, usage, and importance for Physical Design

1 What information does a .v file contain?

- A .v file contains the textual description of a digital design. The content depends on whether it is an RTL .v or a gate-level netlist .v.

Common Elements (mainly in RTL .v)		Additional in Gate-level netlist .v	
Element	Description	Element	Description
module / endmodule	Defines a design block	standard cell instances	e.g. AND2_X1, DFF_X2
input / output / inout	Port declarations	instance names	e.g. U1, U2, u_fft
wire	Net/connection (no memory)	net names	Interconnections between cells
reg	Variable (not always FF)	port connections	Named or positional
parameter	Configurable constants	hierarchical instances	Sub-module instantiations
assign	Continuous combinational logic	tie cells	TIEHI, TIELO
always block	Sequential or combinational logic	buffer/inverter cells	Inserted for timing/DRV
if-else / case	Conditional logic	scan cells	e.g. SDFP, scan logic
generate	Conditional/replicated logic	clock gating cells	e.g. ICG
comments	Design notes (// or /* */)	DFT related logic	scan_en, scan_in/out
		no behavioral constructs	Only structural connectivity

2 How is a .v file used from RTL to GDSII?

The .v file goes through multiple stages in the ASIC flow. Its form changes from RTL (behavioral) to gate-level netlist (structural), but the extension remains .v throughout.

The flowchart illustrates the process from RTL Design (.v) through Simulation & Verification, Synthesis (DC / Genus), Physical Design (Innovus), Signoff (STA, DRC, LVS, etc.), and finally to GDSII (Final layout for fabrication).

3 Difference between Verilog file and netlist?

Both are .v files written in Verilog HDL, but they represent different levels of abstraction.

Feature	Verilog file (RTL)	Netlist (.v)
Who creates	RTL / Front-End designer	Synthesis tool (DC / Genus)
What it describes	What the circuit should do (behavior)	How the circuit is built (structure)
Level of abstraction	Behavioral / RTL	Structural / Technology mapped
Content	always, if-else, case, assign, parameter, etc.	Standard cell instances, net connections
Timing information	No (just intent)	Yes (from .lib)
Example	always @(posedge clk) q <= d;	DFF_X1 U1 (.D(d), .CLK(clk), .Q(q));
Used for	Simulation, Verification, Synthesis	Physical Design, STA, DFT, Signoff
Can Innovus use it directly?	No	Yes
File extension	.v	.v (same)

4 Why should a PD engineer understand a .v file?

Even though PD works mainly with the synthesized netlist, understanding .v files helps in many real-world scenarios.

- Helps to read and debug timing paths**
Instance names, hierarchy, and signal names come from the .v file.
- Useful for debugging synthesis and PnR issues**
Unconnected ports, missing modules, or unexpected logic can be traced back.
- Helps in constraint writing (SDC)**
To identify clocks, resets, and important ports from the .v file.
- Required for ECOs**
Understanding the netlist structure is essential for post-PnR ECOs.
- Important for DFT and scan**
To identify scan chains, scan enable, and test logic.
- Needed for hierarchical design understanding**
To navigate large SoC designs with multiple modules.
- Better communication with RTL and synthesis teams**
Helps in identifying whether an issue is RTL, synthesis, or PD related.
- Essential for signoff and LEC debugging**
To trace logic differences between RTL and netlist.

5 Example: RTL vs Netlist (.v)

RTL .v file (Behavioral)

```

mimo_rx_chain.v
module mimo_rx_chain (
    input wire clk_rf,
    input wire rst_n_rf,
    input wire [15:0] adc_data_i,
    output reg [31:0] rx_data_out,
    output wire rx_overflow
);
always @(posedge clk_rf or negedge rst_n_rf) begin
    if (!rst_n_rf)
        rx_data_out <= 32'd0;
    else
        rx_data_out <= adc_data_i;
end
endmodule
                
```

Describes what the circuit should do (behavioral level)

Gate-level netlist .v file (Structural)

```

mimo_rx_chain_syn.v
module mimo_rx_chain (
    input clk_rf,
    input rst_n_rf,
    input [15:0] adc_data_i,
    output [31:0] rx_data_out,
    output rx_overflow
);
DFF_X1 U1 (.D(n1), .CK(clk_rf), .QN(n2), .RN(rst_n_rf));
AND2_X1 U2 (.A(adc_data_i[0]), .B(n2), .ZN(n3));
...
endmodule
                
```

Describes how the circuit is actually built (structural level with real standard cells)

6 Key Takeaways

- A .v file is a text-based Verilog HDL file.
- Contains design structure, logic, ports, and connections.
- Used throughout the ASIC flow from RTL to GDSII.
- Two main forms: RTL .v and gate-level netlist .v.
- Same extension (.v), different content and purpose.
- PD engineers must understand .v files for debugging, constraint writing, ECOs, and signoff.

7 Common Interview Questions

- What is a .v file?
- What information does a .v file contain?
- How is a .v file used from RTL to GDSII?
- What is the difference between Verilog file and netlist?
- Who creates the .v file?
- Why should a PD engineer understand a .v file?

Remember: A .v file is the designer's intent in text form, which eventually becomes real gates and silicon. | "Same .v extension, different meaning at different stages"

Types of .v Files in ASIC Design

Understand the different forms of .v files you will encounter in the ASIC flow

1 What is RTL Verilog?

- RTL (Register Transfer Level) Verilog is a **text-based** hardware description written by RTL / Front-End designers.
- It describes **what the circuit should do** (functional behavior) using high-level constructs like module, input, output, always, assign, if-else, case, etc.
- It **does not contain actual standard cells**, only design intent.
- It is technology **independent** (can be synthesized to any technology).

Example: RTL Verilog (.v)

```

module adder (
    input wire clk,
    input wire rst_n,
    input wire [7:0] a,
    input wire [7:0] b,
    output reg [7:0] sum
);
always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        sum <= 8'd0;
    else
        sum <= a + b;
end
endmodule
                
```

Key Point: Describes the functionality (what the circuit should do).

2 What is a Gate-Level Netlist?

- A gate-level netlist is a Verilog file generated by **synthesis tools (DC / Genus)**.
- It describes how the circuit is **actually built** using real standard cells from a technology library.
- It contains standard cell instances and their interconnections.
- It is **technology dependent** (mapped to a specific standard cell library).
- It is used for DFT, physical design (PnR), STA, and signoff.

Example: Gate-Level Netlist (.v)

```

module adder (
    input clk,
    input rst_n,
    input [7:0] a,
    input [7:0] b,
    output [7:0] sum
);
DFF_X1 U1 (.D(n1), .CK(clk), .QN(n2));
AND2_X1 U2 (.A(a[0]), .B(b[0]), .ZN(n1));
OR2_X1 U3 (.A(n2), .B(n3), .ZN(sum[0]));
...
endmodule
                
```

Key Point: Describes the structural implementation (using real standard cells).

3 What is a Flat Netlist?

- A flat netlist is a netlist where the entire design is flattened into a **single module**.
- All the sub-modules are removed and their logic is merged into the top module.
- It contains only standard cell instances (no hierarchy).
- It is often used for easier handling in some flows, but it is harder to debug and understand.

Example: Flat Netlist (.v)

```

module top (
    input clk,
    input rst_n,
    input [7:0] in,
    output [7:0] out
);
// All logic from sub-modules is merged here
DFF_X1 U100 (.D(n1), .CK(clk), .QN(n2));
NAND2_X1 U101 (.A(n2), .B(n3), .ZN(n4));
INV_X1 U102 (.A(n4), .ZN(out[0]));
...
endmodule
                
```

Key Point: Single module, no hierarchy, all logic flattened.

4 What is a Hierarchical Netlist?

- A hierarchical netlist **preserves the module hierarchy** from the RTL/synthesis.
- The top module instantiates sub-modules, and each sub-module contains its own logic (standard cells).
- It is easier to debug, understand, and perform ECOs.
- It is commonly used in real projects.

Example: Hierarchical Netlist (.v)

```

module top (
    input clk,
    input rst_n,
    input [7:0] in,
    output [7:0] out
);
u_core core (.clk(clk), .rst_n(rst_n), .in(in), .out(n1));
u_ctrl ctrl (.clk(clk), .rst_n(rst_n), .in(n1), .out(out));
endmodule

module core (
    input clk, input rst_n, input [7:0] in, output [7:0] out
);
// standard cell instances inside core
...
endmodule
                
```

Key Point: Maintains module hierarchy, makes debugging and ECO easier.

5 What is a Technology-Mapped Netlist?

- A technology-mapped netlist is a netlist in which the generic (technology-independent) logic is **mapped to real standard cells** from a specific technology library.
- It is the final netlist after synthesis.
- It contains library-specific standard cell names (e.g., NAND2_X1, DFF_X2).
- It is used for physical design, STA, DFT, and signoff.

Example: Technology-Mapped Netlist (.v)

```

module block (
    input clk,
    input rst_n,
    input a,
    output y
);
NAND2_X1 U1 (.A(a), .B(n1), .ZN(n2));
DFF_X2 U2 (.D(n2), .CK(clk), .QN(y));
...
endmodule
                
```

Key Point: Generic logic is mapped to specific technology library cells.

6 What is an Unmapped Netlist?

- An unmapped netlist is a netlist where the logic is generated after elaboration/optimization but **before technology mapping**.
- It contains generic logic cells (e.g., AND, OR, DFF) instead of real library-specific cells.
- It is technology independent.
- It is an intermediate netlist used inside the synthesis flow.

Example: Unmapped Netlist (.v)

```

module block (
    input clk,
    input rst_n,
    input a,
    output y
);
AND U1 (.A(a), .B(n1), .Z(n2));
DFF U2 (.D(n2), .CK(clk), .Q(y));
...
endmodule
                
```

Key Point: Contains generic cells, not mapped to any technology library.

Understanding Netlists & RTL for Physical Design

Key Concepts: Unmapped Netlist, RTL vs Gate-Level, PD Usage, and Why PnR Needs Synthesized Netlist

1 What is an Unmapped Netlist?

- An unmapped netlist is a netlist where the logic is **generated after elaboration/optimization but before technology mapping**.
- It contains generic logic cells (e.g., AND, OR, DFF, MUX) instead of real technology-specific standard cells.
- It is technology independent.
- It is an intermediate netlist used inside the synthesis flow.

Example: Unmapped Netlist (.v)

```

module block (
    input clk,
    input rst_n,
    input a,
    output y
);
AND U1 (.A(a), .B(n1), .Z(n2));
DFF U2 (.D(n2), .CK(clk), .Q(y));
...
endmodule
                
```

2 Difference between RTL and Gate-Level Netlist

Feature	RTL (Verilog File)	Gate-Level Netlist (.v)
Who creates	RTL / Front-End designer	Synthesis tool (DC / Genus)
Describes	What the circuit should do (functional behavior)	How the circuit is actually built (structure)
Level of abstraction	Behavioral / RTL	Structural (technology mapped)
Content	High-level constructs: module, always, if-else, assign	Standard cell instances, net connections

```

module block (
  input clk,
  input rst_n,
  input a,
  output y
);
AND U1 ( .A(a), .B(n1), .Z(n2) ); // generic cell
DFF U2 ( .D(n2), .CK(clk), .Q(y) ); // generic cell
endmodule

```

Contains generic cells
(not mapped to any
technology library)

Key Point: Unmapped netlist has generic cells and is converted to a technology-mapped netlist in the next step.

	module, always, if-else, assign, case, parameters, etc.	net connections
Cells used	No real standard cells	Real technology-specific standard cells (e.g., NAND2_X1)
Timing information	No (just intent)	Yes (from .lib)
Example	always @(posedge clk) q <= d;	DFF_X1 U1 (.D(d), .CK(clk), .Q(q));
Used for	Simulation, Verification, Synthesis	Physical Design (PnR), STA, DFT, Signoff

3 Which .v file is directly used by a PD engineer?

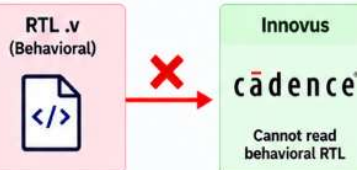
- A PD engineer directly uses the **gate-level netlist (.v)** generated by the synthesis tool (DC / Genus).
- This netlist contains real standard cells from the technology library with proper connectivity.
- PD engineers may also look at RTL .v files for debugging, understanding hierarchy, ports, and logic, but for PnR, the gate-level netlist is used.

Used in Physical Design Flow



4 Can Innovus read RTL directly?

- No**, Innovus **cannot read behavioral RTL directly** for PnR.
- Innovus expects a synthesized gate-level netlist with technology-specific standard cells.
- It can read Verilog only in structural (gate-level) form, not behavioral RTL with high-level constructs like always, if-else, assign, etc.



5 Why can't PnR directly use behavioral RTL?

- Behavioral RTL describes what the circuit does, not how it is built.
- It does not contain real standard cells, net connectivity, or physical information.
- PnR tools (like Innovus) need actual cells with dimensions, timing, and physical views (LEF/DEF).
- Behavioral constructs (always, if-else, case, etc.) must be converted into a structural netlist through synthesis.
- Without synthesis, PnR has no information about the actual gates, cell sizes, and connectivity.



Key Takeaway: RTL .v describes the intent, but Physical Design works with the synthesized gate-level netlist.

"Design in RTL, Implement in Gates, Place and Route to Silicon"

Key RTL Constructs in Verilog (for PD Engineer)

Understand the essential RTL constructs you will see in a .v file

1 Constructs present in RTL

- RTL (Verilog) uses various constructs to describe the functionality of a digital design.
- Common constructs:
 - module / endmodule
 - input / output / inout
 - wire, reg
 - parameter
 - assign
 - always block (sequential or combinational)
 - if-else, case
 - posedge / negedge
 - reset
 - generate
- These constructs describe **what** the circuit should do (behavior).

2 module and endmodule

- module** defines a design block with a name, ports and internal logic.
- endmodule** marks the end of the design block.

Example:

```

module adder (
  input wire a,
  input wire b,
  output wire sum
);
  assign sum = a + b;
endmodule

```



A module is a building block.
endmodule ends the block.

3 input / output / inout ports

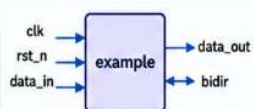
- Ports are used to communicate signals with the outside world.
- input** : signal comes into the module
- output** : signal goes out of the module
- inout** : bidirectional signal

Example:

```

module example (
  input wire clk,
  input wire rst_n,
  input wire [7:0] data_in,
  output wire [7:0] data_out,
  inout wire bidir
);
  ...
endmodule

```



4 wires and registers

- wire** : represents a physical connection (net). No memory. Driven by continuous assignment or module output.
- reg** : represents a variable that stores a value. Used in always blocks. Not always a flip-flop (it depends on the logic).

Example:

```

wire a, b, y; // wires
reg q; // register

assign y = a & b;

always @(posedge clk) begin
  q <= y;
end

```

wire = connection (no memory)
reg = variable (can infer flip-flop or latch depending on logic)

5 assign statement

- assign** is a continuous assignment used to model combinational logic.
- It is always active and evaluated whenever input changes.

Example:

```

wire a, b, y;
assign y = a & b;
assign sum = a + b;

```



assign is used for combinational logic.

6 always block

- always** block describes the behavior of logic.
- It can be sequential (with clock) or combinational (without clock).

Example (Sequential):

```

always @(posedge clk) begin
  q <= d;
end

```

Example (Combinational):

```

always (*) begin
  y = a & b;
end

```

always block is used to describe sequential or combinational logic.

7 Sequential logic

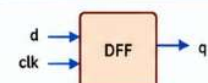
- Sequential logic** has a clock and stores state (uses flip-flops).
- Output changes on clock edge.

Example:

```

always @(posedge clk) begin
  if (!rst_n)
    q <= 1'b0;
  else
    q <= d;
end

```



Sequential logic = clocked logic
(uses flip-flops).

8 Combinational logic

- Combinational logic has no clock.
- Output depends only on current inputs.
- No memory elements.

Example:

```

always (*) begin
  y = (a & b) | c;
end

```



Combinational logic =
no clock, no memory
(just gates).

9 posedge and negedge

- posedge** : rising edge of a clock (signal goes from 0 → 1).
- negedge** : falling edge of a clock (signal goes from 1 → 0).

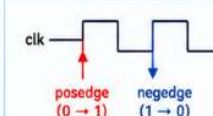
Example:

```

always @(posedge clk) begin
  q <= d;
end

always @(negedge clk) begin
  q <= d;
end

```



10 Reset

- Reset is used to initialize the design to a known state.
- It can be synchronous (works with clock) or **asynchronous** (works independent of clock).

Example (Asynchronous Reset):

```

always @(posedge clk or
  negedge rst_n) begin
  if (!rst_n)
    q <= 1'b0;
  else
    q <= d;
end

```

Reset ensures the circuit starts from a known state.



Key Takeaway: RTL constructs describe the behavior (what the circuit should do). Synthesis converts this into real gates using a technology library.

"Understand RTL, Debug Better in PD"

How to Read .v File as PD Engineer

Understand the netlist, identify key elements, and use Innovus to explore the design

1 Read Netlist and Identify Top Module & Submodules

How to read a netlist?

- A gate-level netlist is a structural Verilog file containing standard cell instantiations.
- Start from the top module and understand the ports and instantiated submodules.
- Check the hierarchy to see how modules are connected.

Example: Netlist snippet

```
// Top module
module chip_top (
    input clk,
    input rst_n,
    input [7:0] in,
    output [7:0] out
);
    // Submodule instantiations
    u_cpu cpu_top (
        .clk(clk),
        .rst_n(rst_n),
        .in(in), .out(cpu_out);
    u_mem mem_ctrl (
        .clk(clk),
        .rst_n(rst_n),
        .data(cpu_out),
        .mem_data(out) );
endmodule
```

In Innovus

```
# Set and check top module
set_db init_design_set_top chip_top
get_db design
get_db design .top

# View hierarchy
get_db insts -hier
get_db insts u_cpu
get_db insts u_mem
```

Top module is the starting point for PnR. Submodules are instantiated inside the top.

2 Identify Clocks, Ports, Resets, Scan Ports and Power Related Cells

1. Ports

- Check module port list (input/output/inout).
- Identify important ports like clocks, resets, scan signals, power pins, etc.

```
module chip_top (
    input clk,
    input rst_n,
    input scan_en,
    input scan_in,
    output scan_out,
    input VDD,
    input VSS
);
```

Ports are defined in the top module declaration.

2. Clocks

- Search for clock ports (clk, clock).
- Look for posedge/negedge usage.
- Check for multiple clocks.

```
always @(posedge clk) begin
    q <= d;
end
```

Clock can be identified by name and by its usage in always blocks.

3. Resets

- Search for reset ports (rst, rst_n).
- Check if it's active high or low.

```
always @(posedge clk or
negedge rst_n) begin
    if (!rst_n)
        q <= 'b0;
    else
        q <= d;
end
```

Reset is identified by name and its usage in always block.

4. Scan Ports (DFT)

- Look for scan related ports and signals.
- Common names: scan_en, scan_in, scan_out, scan_clk, SI, SO, SE.

```
input scan_en,
input scan_in,
output scan_out,
input scan_clk
```

Scan ports are used for DFT testing.

5. Power Related Cells

- Search for tie cells, level shifters, isolation cells, etc.
- Common cell names: TIEH_X1, TIELO_X1, ISO_X1, LS_X1, RET_X1, etc.

```
TIEH_X1 U_TIE1 ( .ZN(vdd_net) );
TIELO_X1 U_TIE0 ( .ZN(gnd_net) );
ISO_X1 U_ISO ( .A(sig_in), .Y(sig_out), .ISO_EN(iso_en) );
```

Power related cells are easily identified by their cell names.

3 Identify ICG Cells, Memories/Macros, Black Boxes, Hierarchy and Cell Instances

1. ICG Cells (Integrated Clock Gating)

- Used to gate the clock based on enable signal.
- Common names: ICG_X1, CLK_GATE_X1, etc.

```
ICG_X1 U_ICG (
    .CLK(clk),
    .EN(en),
    .Z(gated_clk)
);
```

ICG cells help in reducing dynamic power.

2. Memories / Macros

- Large blocks like SRAM, ROM, PLL, etc.
- They may be provided as black boxes (without internal logic).
- Identified by their module names.

```
SRAM_1P u_sram (
    .CLK(clk), .A(addr), .D(din),
    .Q(dout), .CS(cs), .WE(we)
);
```

Macros need LEF/Lib for physical implementation.

3. Black Boxes

- Modules without internal definition in the netlist.
- Usually third-party IP or hard macros.

```
module usb_phy (
    input clk,
    input rst_n,
    // no internal logic
);
```

Treated as a black box during PnR.

4. Hierarchy

- Submodules are instantiated inside the top or other modules.
- Use tools to view the hierarchy tree.

```
chip_top
├── u_cpu (cpu_top)
│   ├── u_alu (alu)
│   ├── u_reg (regfile)
│   └── u_mem (mem_ctrl)
└── u_sram (SRAM_1P)
```

Helps in block-level PnR and debugging.

5. Cell Instances

- Each standard cell is instantiated with a unique instance name.
- Format: <cell_name>-<instance_name>-<pin>.

```
DFF_X1 U1 (.D(d), .Q(q), .Q(q));
AND2_X1 U2 (.A(a), .B(b), .Z(z));
```

Instance names (e.g., U1, U2) identify each cell in the design.

4 Useful Innovus Commands for Exploration

Task	Innovus Command	Description
Read netlist	read_verilog chip_top.v	Read the .v file
Set top module	set_db init_design_set_top chip_top	Set top design
Check current design	get_db design	Show design info
Get top	get_db design .top	Show top module
List all instances	get_db insts	List instances (top level)
List hierarchical instances	get_db insts -hier	List all instances (hierarchy)
List ports	get_db ports	Show all ports
List clocks	get_db ports *clk*	Find clock ports
List resets	get_db ports *rst*	Find reset ports
List scan ports	get_db ports *scan*	Find scan related ports
Find ICG cells	get_db insts -hier -filter {ref_name == ICG*}	List ICG instances
Find memory/macros	get_db insts -hier -filter {ref_name == SRAM*}	List memory instances
Find black boxes (unresolved modules)	check_design -unresolved	List unresolved modules
Find power related cells	get_db insts -hier -filter {ref_name == TIE* ref_name == ISO* ref_name == LS*}	List power related cells
View hierarchy tree	report_design_hierarchy	Show hierarchy
More information about an instance	get_db insts U1	Show details of instance U1



Key Takeaway: A .v netlist may look complex, but it is just a structural description. Start from the top module, identify important ports, clocks, resets, scan signals, special cells and hierarchy. Use Innovus commands to quickly explore and understand the design.

"Read the netlist, understand the design, implement with confidence."

First Thing a PD Engineer Does After Receiving a .v File

A step-by-step flow to check, understand and prepare the design for implementation

Read → Analyze → Verify → Load into Tool → Ready for PnR

1 Complete Flow Overview

- Receive .v File**
Get the netlist/RTL and related files from frontend.
- Check File Availability**
Verify all required files are available (.v, .lib, .lef, .sdc, etc.).
- Identify File Type**
Check if it is RTL or Gate-level netlist.
- Find Top Module**
Identify the top design module.
- Check Hierarchy**
Understand submodules and instances.
- Check Clocks**
Identify all clocks (primary, generated, gated).
- Check Resets**
Identify reset signals and type (sync/async).
- Check Ports**
List input, output, inout and special ports (scan, test, etc.).
- Check Black Boxes / Macros**
Identify memories, hard macros and unresolved modules.
- Check DFT / Scan**
Identify scan chains, scan ports and DFT logic.
- Check Power Intent**
Look for power related cells (tie, ISO, level shifters, etc.).

2 Check File Availability

- Ensure you have all necessary files.
- Typical files:

```
design.v (netlist/RTL)
*.lib (timing library)
*.lef (physical library)
*.sdc (constraints)
*.tf / *.spef (optional)
```

3 Identify File Type

- Check if the file is RTL or Gate-level netlist.
- In PD, you usually receive a gate-level netlist.

Example:

```
grep -rn "module" design.v
```

If you see **always**, **if**, **case** → it is RTL.
If you see only standard cell instantiations → it is gate-level.

4 Find Top Module

- Top module is the starting point for PnR.
- Use grep or tool commands.

Example (Linux):

```
grep -rn "module" design.v
# Check which module is the top
```

Example (Innovus):

```
get_db init_design_set_top chip_top
get_db design .top
```

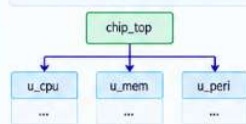
Make sure the correct top module is selected.

5 Check Hierarchy

- List all submodules and instances.
- Helps to understand block structure.

Example (Innovus):

```
get_db insts -hier
get_db insts u_sub_block -hier
```

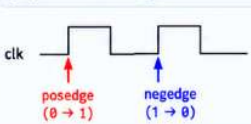


6 Check Clocks

- Identify all clock ports and internally generated clocks.
- Look for clock names and always blocks.

Example (Linux):

```
grep -r "clk" design.v
```



7 Check Resets

- Identify reset signals (e.g., rst, rst_n).
- Check if reset is synchronous or asynchronous.

Example (Linux):

```
grep -r "rst" design.v
```

```
always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        q <= 'b0;
    else
        q <= d;
end
```

8 Check Ports

- List all input, output, inout ports.
- Check for special ports (scan, test, etc.).

Example (Innovus):

```
get_db ports
```

9 Check Black Boxes / Macros

- Identify memories, hard macros and unresolved modules.
- These will need LEF/physical information for PnR.

Example (Innovus):

```
get_db insts
```

10 Check DFT / Scan

- Look for scan ports and DFT related logic.
- Common names: scan_in, scan_out, scan_en, test_mode, SI, SO, SE.

Example (Linux):

```
grep -r "scan" design.v
```

level shifter, etc.).

12 **Read into Tool**

Read the netlist in Innovus (or DC/Genus).

13 **Elaborate / Link**

Resolve references and link the design.

14 **Run Design Checks**

Check for missing modules, ports, etc.

15 **Ready for Next Stage**

Design is ready for floorplanning and PnR.

11 **Check Power Intent**

- Identify power related cells.
- Common cells: TIEHI, TIELO, ISO, LS (level shifter), RET (retention), etc.

Example (Linux):

```
grep -r "TIE\|ISO\|LS\|RET" design.v
```

12 **Read into Tool and Link**

- Read the netlist in Innovus.
- Link the design and check for errors.

Example (Innovus):

```
read_verilog design.v
set_db init_design_set_top chip_top
link_design
check_design
```

13 **Run Design Checks**

- Check for missing modules, unconnected ports, etc.

Example (Innovus):

```
check_design
report_design
report_hierarchy
report_timing
```

Key Takeaway: Do a systematic check of the .v file before starting implementation. It saves time and avoids major issues during PnR.

"Understand the design first, implement with confidence."

Linux Commands in Initial .v File Investigation (Part 2/2)

Find always blocks, assign statements, scan signals, non-synthesizable constructs and compare netlists

>_

Explore Search Analyze Understand

6 **Find always Blocks and assign Statements**

Useful to understand RTL style and logic in the design.

Command	Purpose
grep -rn "always" <dir>	Find all always blocks
grep -rn "assign" <dir>	Find all assign statements
grep -rn "always @(posedge)" <dir>	Find sequential always blocks
grep -rn "always @(negedge)" <dir>	Find negedge always blocks

Example: always block

```
always @(posedge clk) begin
    q <= d;
end
```

Example: assign statement

```
assign y = a & b;
```

7 **Find Scan Signals**

Useful to identify scan related ports and signals (DFT).

Command	Purpose
grep -rn "scan" <dir>	Find scan related signals
grep -rn "scan_in" <dir>	Find scan_in signals
grep -rn "scan_out" <dir>	Find scan_out signals
grep -rn "scan_en" <dir>	Find scan enable signals
grep -rn "SI" <dir>	Find SI (Scan In)
grep -rn "SO" <dir>	Find SO (Scan Out)

Common Scan Signal Names

- scan_in / SI
- scan_out / SO
- scan_en / SE
- test_mode
- scan_clk
- scan_rst

Example: Scan Port Declaration

```
module top (
    input scan_in,
    input scan_en,
    output scan_out
);
```

8 **Find Non-Synthesizable Constructs**

Useful to find constructs that are not synthesizable (used in testbenches or for simulation only).

Command	Purpose	Common Examples
grep -rn "\$display" <dir>	Find \$display statements	\$display, \$write, \$monitor
grep -rn "\$finish" <dir>	Find \$finish statements	\$finish
grep -rn "#" <dir>	Find delays (#)	#10, #100
grep -rn "initial" <dir>	Find initial blocks	initial begin ... end
grep -rn "fork" <dir>	Find fork-join	fork ... join
grep -rn "wait" <dir>	Find wait statements	wait(signal)

Example: initial block

```
initial begin
    $display("Simulation done");
end
```

Example: fork-join

```
fork
    assign y = a & b;
join
```

These are not synthesizable!
They are typically used in testbenches or for simulation and should not be present in synthesizable RTL.

9 **Compare Two Netlists**

Useful to compare two versions of netlists (e.g., pre and post ECO).

Command	Purpose	When to Use?
diff file1.v file2.v	Show line-by-line differences	After ECO changes
diff -u file1.v file2.v	Show differences in unified format	To compare synthesis outputs
diff -q file1.v file2.v	Quick check (files are same or not)	To check if two netlists are identical
diff -q dir1 dir2	Compare directories recursively	To debug unexpected differences

Example: diff command

```
diff file1.v file2.v
```

Example: diff -u command

```
diff -u file1.v file2.v
```

Example: diff -q command

```
diff -q file1.v file2.v
```

Example: diff -q command

```
diff -q dir1 dir2
```

Tips:

- Use diff -q for a quick check.
- Use diff -u to see exact changes.
- You can also use diff -r for directory comparison.

Key Takeaway: These Linux commands help you quickly explore, understand and verify the .v file before loading it into tools. Use them regularly to save time and debug issues early.

"A few simple commands can reveal a lot about your design."

Linux Commands & Clock Analysis in .v File Investigation

Practical examples for PD Engineers

>_

Explore → Analyze → Understand → Save Time

1 **How to List all .v Files ?**

Use find or ls commands to list all Verilog files in the current directory or subdirectories.

Commands

```
find . -name "*.v" # List all .v files (recursive)
find rtl -name "*.v" # In specific directory
ls *.v # List .v files (current dir only)
ls -l *.v # One file per line
```

Example

```
user@pd:~/design$ find . -name "*.v"
./top.v
./rtl/alu.v
./rtl/mem_ctrl.v
./rtl/uart.v
```

Tip: Use find for large designs as it searches in all subdirectories.

2 **How to Count .v Files ?**

Count the total number of Verilog files.

Commands

```
find . -name "*.v" | wc -l # Count all .v files
ls *.v | wc -l # Count in current dir
```

Example

```
user@pd:~/design$ find . -name "*.v" | wc -l
128
```

How to Check File Size ?

Commands

```
ls -lh <file> # Human readable size
du -sh <file> # Total size
du -sh <dir> # Directory size
```

Example

```
user@pd:~/design$ ls -lh top.v
-rw-r--r-- 1 user user 4.5M May 10 10:21 top.v
user@pd:~/design$ du -sh top.v
4.5M top.v
```

3 **How to Compare Two Netlists ?**

Use diff command to compare two versions of a netlist.

Commands

```
diff file1.v file2.v # Show differences (line by line)
diff -u file1.v file2.v # Unified format (recommended)
diff -q file1.v file2.v # Quick check (same or different)
diff -r dir1 dir2 # Compare directories
cmp file1.v file2.v # Simple comparison
```

Example

```
user@pd:~/design$ diff -u netlist1.v netlist2.v
--- netlist1.v 2025-05-10 10:00:00
+++ netlist2.v 2025-05-12 11:00:00
@@ -10,7 +10,7 @@
- assign y = a & b;
+ assign y = a | b;
```

Example

```
user@pd:~/design$ diff -q netlist1.v netlist2.v
Files netlist1.v and netlist2.v differ
```

Tip: Use diff -u to clearly see what has changed between two netlist versions (e.g., pre and post ECO).

4 **How to Identify Clocks from RTL ?**

Clocks are usually used in always blocks with posedge or negedge.

Typical Clock Definition

```
always @(posedge clk) begin
    q <= d;
end
```

Meaning of posedge and negedge

Term	Meaning
posedge	Trigger on rising edge (0 → 1)
negedge	Trigger on falling edge (1 → 0)

Waveform

5 **How to Identify Multiple Clock Domains, Gated Clocks and Clock Dividers ?**

Check for different clock signals, clock gating logic and clock divider logic in RTL.

1. Multiple Clock Domains

- A design can have more than one clock signal (e.g., clk, clk2, clk_a, etc.).
- Search for different clock names in the code.

Example:

```
always @(posedge clk) begin
    ...
end
always @(posedge clk2) begin
    ...
end
```

2. Gated Clock

- A gated clock is generated by enabling/disabling the clock using a control signal (ICG cell or logic).
- Used for power saving.

Example (RTL style):

```
assign gclk = clk & en;
```

3. Clock Divider

- A clock divider generates a lower frequency clock

https://onedrive.live.com/personal/938043310494b6ef/_layouts/15/Doc.aspx?sourcedoc={0494b6ef-4331-2080-8093-7a0000000000}&action=edit&w...

5/14

Find Clocks in Files (Linux)

```
grep -rn "posedge" . # Find all posedge clocks
grep -rn "negedge" . # Find all negedge clocks
grep -rn "clk" . # Search by clock name
```

```
user@pd:~/design$ grep -rn "posedge" ./rtl
./rtl/alu.v:12: always @(posedge clk) begin
```

clock domains.

Find Different Clocks in Linux

```
grep -rn "always @(posedge" . # Find posedge clocks
grep -rn "always @(negedge" . # Find negedge clocks
grep -rn "clk[a-zA-Z0-9_]*" . # List all clock names
```

```
user@pd:~/design$ grep -rn "clk[a-zA-Z0-9_]*" . | sort -u
clk
clk2
clk_div
gclk
icg_clk
```

by dividing the input clock.

- Usually implemented using a counter or dedicated cell.

Example:

```
always @(posedge clk) begin
  if (count == 1) begin
    div_clk <= ~div_clk;
    count <= 0;
  end else begin
    count <= count + 1;
  end
end
```

clk (100 MHz) → ÷2 → div_clk (50 MHz)



Key Takeaway: Linux commands help you quickly explore the .v file, compare versions, and identify important design details like clocks, clock domains, gated clocks and clock dividers. This saves time and helps you understand the design before implementation.

"Explore the file, understand the design, implement with confidence."

.v Clean RTL Leads to Better Silicon

RTL Checks Before Synthesis

Catch issues early • Clean RTL • Smooth Synthesis • Better Quality Design

Check Early
Save Time
Build Better Chips!

1 Check All Files

- Ensure all required RTL files are available and accessible.
- Verify no file is missing or empty.
- Check file count and size.

Useful Commands

```
ls *.v
find . -name "*.v" | wc -l
ls -lh
```

Tip: Make sure you have the complete RTL (drop) from the frontend team.

2 Check Top Module

- Identify the correct top module (starting point for synthesis).
- Ensure only one top module is used.
- Verify it matches the project specification.

Useful Commands

```
grep -rn "module" .
grep -rn "module <top_name>" .
```

Tip: Top module is usually the highest level module (not instantiated by any other module).

3 Check Clocks

- Identify all clock signals and their names.
- Check for posedge/negedge usage.
- Look for multiple clock domains.

Useful Commands

```
grep -rn "posedge" .
grep -rn "negedge" .
grep -rn "clk" .
```

Tip: Different clock names indicate different clock domains. Verify with design spec.

4 Check Resets

- Identify reset signals (e.g., rst, rst_n).
- Check if reset is synchronous or asynchronous.
- Verify proper reset polarity and connection.

Useful Commands

```
grep -rn "rst" .
grep -rn "reset" .
```

Tip: Confirm reset strategy with design spec (e.g., active high/low, sync/asyn).

5 Check Non-Synthesizable Code

- Find constructs not supported by synthesis tools (e.g., \$display, #delay, initial, fork-join).
- These are allowed in testbench, not in synthesizable RTL.

Useful Commands

```
grep -rn "\$display" .
grep -rn "#:" .
grep -rn "initial:" .
grep -rn "fork:" .
```

Tip: Remove or wrap non-synthesizable code using `ifdef SYNTHESIS`.

6 Check Latch Risks

- Check for incomplete if/else or case statements which can infer latches.
- Ensure all signals are assigned in all conditions.

Useful Commands

```
grep -rn "always" .
grep -rn "if (" .
grep -rn "case" .
```

Tip: Make sure every output is assigned in all paths to avoid unintended latches.

7 Check Combinational Loops

- Ensure there are no combinational loops, which can cause synthesis and timing issues.
- Check for feedback paths without sequential elements.

Useful Commands (Basic Check)

```
grep -rn "assign" .
```

(Manual review / EDA lint tools for accurate check)

Tip: Use lint tools or synthesis tool checks (e.g., check_design in DC/Genus) for reliable detection.

8 Check Multiple Drivers

- Ensure no signal is driven by more than one source (except tri-state/bus structures).
- Multiple drivers can cause functional and synthesis errors.

Useful Commands

```
grep -rn "assign" .
grep -rn "output" .
```

Tip: Check especially for signals driven in multiple always blocks or modules.

9 Check Undriven Signals

- Ensure all signals are properly driven.
- Undriven signals can lead to X values and synthesis warnings.

Useful Commands (Basic)

```
grep -rn "wire" .
grep -rn "reg" .
```

(Use lint/synthesis tool for accurate check)

Tip: Use lint tools or check_design in synthesis tool to find undriven nets.

10 Check Black Boxes

- Identify modules without implementation (e.g., memories, hard macros, third-party IP).
- Ensure proper .v, .lib and .lef files are available.

Useful Commands

```
grep -rn "module" . | grep -v "{"
(List module definitions)
Check synthesis tool reports for unresolved references.
```

Tip: Black boxes must have proper libraries and physical views for later stages.

11 Check Hierarchy

- Understand the module hierarchy.
- List all submodules and instances.
- Verify instance connections and naming.

Useful Commands

```
grep -rn "module" .
grep -rn "w[0-9a-zA-Z_]*s[0-9a-zA-Z_]*s[0-9a-zA-Z_]*" .
(Find instances)
Use synthesis tool: report_hierarchy
```

Tip: Ensure hierarchy matches the design specification.

12 Check High Fanout Nets

- Identify nets that drive a large number of loads.
- High fanout can impact timing and may need buffering.

Useful Commands (Approximate)

```
grep -rn "\b<signal_name>b" .
```

(Manual check) Use synthesis tool (report_high_fanout).

Tip: Use tool reports for accurate high fanout information.

13 Check Missing Modules

- Ensure all instantiated modules have RTL or library files.
- Check for unresolved references.
- Missing modules will cause synthesis to fail.

Useful Commands

```
grep -rn "module" .
```

Use synthesis tool: check_design / report_unresolved

Tip: Provide all required RTL and library files (including IPs and macros) before synthesis.



Key Takeaway: Performing these RTL checks before synthesis helps catch issues early, improves design quality, reduces iteration time, and ensures a smooth synthesis and implementation flow.

"Clean RTL today, Faster tapeout tomorrow."

RTL to Silicon

Latch in RTL Design

Concept | Inference | Why it's Dangerous | Detection | Impact on STA | Intentional Latch

Understand Latches
Avoid Unintentional Latches
Design Reliable Silicon

1 What is a Latch ?

- A latch is a level-sensitive storage element that holds data when enabled and is transparent when enable is active.
- Unlike flip-flops (edge-triggered), latches are transparent for a level of the clock/enable.
- It can store 1 bit of information.

D → [Latch] → Q

↑

EN

EN	Q(next)
1	D (transparent)
0	Q (holds)

Latch = Level Sensitive
Flip-Flop = Edge Sensitive

2 How is Latch Inferred ?

- A latch is inferred when a variable is not assigned in all possible conditions of a combinational block.
- Common causes:
 - Missing else branch
 - Missing default assignment
 - Incomplete case statement
 - Conditional assignment
 - Partially assigned signals/bit slices

Example: Latch Inference

```
always @(*) begin
  if (sel)
    y = a;
  // missing else → latch inferred
end
```

When sel = 0, "y" is not assigned, so it must hold its previous value → Latch inferred.

3 Why is Latch Dangerous ?

- Latches are usually unintended in RTL.
- Can create transparent paths and race conditions.
- May cause functional bugs if not properly handled.
- Difficult to analyze and verify.
- Can lead to timing issues and unstable behavior across PVT corners.
- May break synchronous design methodology.
- Can create combinational loops if used incorrectly.

Unintended latches can make the design unpredictable and hard to debug.

4 How to Detect Latch ?

- Check synthesis tool reports (recommended).
- Look for keywords: Latch, LATCH, level-sensitive.
- Use lint/CDC/RTL check tools.
- Manually review always @(*) blocks for incomplete assignments.
- Check case statements for missing default.

Example: Tool Report (Synopsys DC)

Warning: Latch inferred for signal 'y' in module 'block1'.
LATCH y (location: rtl.v:21)



Always fix unintended latches before synthesis.

5 Impact on STA ?

- Latches are level-sensitive, so STA must model time borrowing which is more complex than edge-triggered flip-flops.
- Creates two-phase timing (latch open/close).
- Can introduce hold time violations due to transparency.
- Makes timing analysis and closure harder.
- Can reduce maximum frequency.
- May require special constraints (e.g., set_clock_gating_check, set_ideal_latency).



Unintended latches make STA more complex and can cause hold violations and timing instability.

6 What is an Intentional Latch ?

- A latch can be intentionally used in specific designs, for example:
 - Low-power designs (time borrowing)
 - Two-phase latch-based designs
 - Asynchronous interfaces
 - Level-sensitive pipeline stages
 - Special IP or library-based latch cells

Example: Intentional Latch RTL

```
always_latch begin
  if (en)
    q <= d;
end
```

Here, a latch is explicitly intended using always_latch (SystemVerilog).

7 How to Avoid Unintended Latches ?

- Provide default assignment for all signals.
- Use complete if-else or case with default.
- Use always_comb instead of always @(*) (SystemVerilog).
- Run lint and synthesis checks.
- Review all combinational blocks carefully.

Correct Coding Style (No Latch)

```
always @(*) begin
  y = 1'b0; // default assignment
  if (sel)
    y = a;
  else
    y = b;
end
```

8 Summary

Aspect	Key Points
What	Level-sensitive storage element.
How inferred	Incomplete assignments in combinational logic.
Why dangerous	Can cause functional and timing issues.
Detection	Synthesis reports, lint tools, manual review.
Impact on STA	More complex timing, possible hold violations.
Intentional latch	Used in low-power, two-phase or special designs.



Key Takeaway:

Latches are powerful but must be used carefully. Avoid unintended latches, fix them before synthesis, and understand their impact on STA. Use latches intentionally only when required by the design architecture.

"Clean RTL → Clean Synthesis
→ Reliable Silicon"



RTL to
Synthesizable
Hardware

Non Synthesizable Constructs

Understand What Can Be Synthesized and What Cannot

"Write Synthesizable RTL
for Real Silicon,
Use Non-Synthesizable
Only for Verification!"

1 What is Synthesizable RTL ?

- Synthesizable RTL is a subset of Verilog/SystemVerilog that can be converted into a real hardware implementation (gates, flip-flops, multiplexers, etc.) by a synthesis tool.
- It describes hardware behavior using constructs that have a clear hardware meaning.
- It follows coding guidelines and avoids simulation-only features.

Example: Synthesizable RTL

```
module counter (
  input clk,
  input rst_n,
  output reg [7:0] count
);
always @(posedge clk or negedge
rst_n) begin
  if (!rst_n)
    count <= 8'd0;
  else
    count <= count + 1;
end
endmodule
```

Synthesizable Constructs Example:

- module
- input/output
- always @(posedge/negedge)
- assign
- if/else, case
- for (with constant bounds)
- parameters
- generate

2 What is Non-Synthesizable RTL ?

- Non-synthesizable RTL contains constructs that cannot be mapped to real hardware by a synthesis tool.
- These constructs are mainly used for simulation, verification, and testbenches.
- They do not have a direct hardware equivalent or have ambiguous/delay-based behavior.

Example: Non-Synthesizable Constructs

```
module tb;
  initial begin
    $display("Hello");
    #10;
    $finish;
  end
endmodule
```



Common Non-Synthesizable Constructs:

- initial blocks (for initialization, stimulus)
- delay statements (#)
- \$display, \$monitor, \$finish, \$stop
- file I/O (\$fopen, \$fwrite, etc.)
- fork...join
- forever (without hardware meaning)
- wait, force, release
- randomization (\$random)
- testbench constructs

3 What Happens if Synthesis Sees Non-Synthesizable Code ?

- The synthesis tool will either:
 - Report an error (most common), or
 - Ignore the construct (with a warning).
- Non-synthesizable code will not be converted into hardware.
- This can lead to:
 - Missing functionality in the synthesized design
 - Mismatch between simulation and hardware
 - Unexpected behavior after implementation

Example: Tool Message (Synopsys DC)

Warning: Ignoring non-synthesizable construct 'initial' (rtl.v:12)
Error: Unsupported system task '\$display' (rtl.v:15)



Always check synthesis logs and fix or remove non-synthesizable constructs before synthesis.

4 Common Non-Synthesizable Constructs and Their Meaning

Construct	Description	Example	Use Case	Synthesizable?
initial	Used for initialization or stimulus	initial begin a = 0; end	Testbench / Simulation	✗ No
# (delay)	Specifies time delay	#10 a = 1;	Testbench / Timing control	✗ No
\$display	Prints text to console	\$display("Value = %d", a);	Debug / Simulation	✗ No
\$monitor	Continuously prints when signal changes	\$monitor("time=%t a=%d", \$time, a);	Debug / Simulation	✗ No
\$finish	Ends simulation	\$finish;	Testbench	✗ No
\$stop	Stops simulation (interactive)	\$stop;	Testbench	✗ No
fork...join	Used for parallel processes	fork ... join	Testbench	✗ No
wait	Wait for a condition	wait(a == 1);	Testbench	✗ No
force / release	Forces a signal value	force a = 1;	Testbench	✗ No
\$fopen, \$fwrite	File I/O operations	\$fopen("file.txt", "w");	Testbench / Logging	✗ No
\$random	Generates random numbers	a = \$random;	Testbench / Random stimulus	✗ No



Key Takeaway: Use only synthesizable constructs in RTL meant for synthesis. Keep non-synthesizable constructs in testbenches or verification code. Always run lint and synthesis checks to catch unsupported constructs early.

Synthesizable RTL builds hardware, Non-synthesizable RTL verifies it."



From RTL
to
Silicon

Netlist Hierarchy vs Flat Netlist

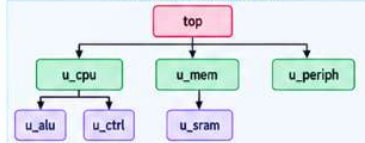
Same Logic | Different Views | Different Impacts

Understand Hierarchy
Place Better
Analyze Faster
Design Smarter

1 What is a Hierarchical Netlist ?

- A hierarchical netlist preserves the module hierarchy from RTL.
- The design is described in multiple levels (top module → submodules → sub-submodules).
- Each module is kept as a separate block with its own instances and connections.

Hierarchical Netlist (Example)

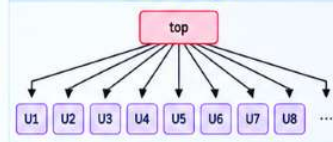


- Easier to read and debug
- Preserves design structure
- Commonly used in modern SoC/complex designs

2 What is a Flat Netlist ?

- A flat netlist has no hierarchy.
- All modules are flattened into a single top-level module with no submodules.
- All instances are at the same level.

Flat Netlist (Example)



- No design hierarchy
- Everything in one level
- Usually generated by some synthesis flows (optional)
- Can be harder to debug for large designs

3 Hierarchy vs Flat Netlist – Comparison

Aspect	Hierarchical Netlist	Flat Netlist
Structure	Preserves module hierarchy	No hierarchy (single level)
Readability	Easier to understand	Difficult for large designs
File size	Smaller (modular)	Larger
Debugging	Easier (module-wise)	Harder
Tool runtime	Faster for large designs	Slower (more instances)
Optimization	Allows block-level optimization	Global optimization (possibly better, but at cost)
Placement	Can use hierarchical floorplan	No module boundaries
STA	Hierarchical analysis (faster, scalable)	Full flat analysis (slows down for big designs)
Use case	SoC / large block designs	Small designs or final signoff (flattened netlist)

4 How Does Hierarchy Impact Placement ?

- Hierarchy allows block-level floorplanning (place each module in a specific region).
- You can preserve module boundaries using placement constraints.
- Helps in managing large designs (e.g., CPU, memory, peripherals).
- Flat netlist gives no natural boundaries → harder to control placement and congestion.

Hierarchical Placement



Module-wise placement (easier to manage)

Flat Placement



No module boundaries (harder to control)

5 How Does Hierarchy Impact STA ?

- Hierarchical netlist:**
 - Faster analysis for large designs (block-wise).
 - Allows hierarchical STA (analyze block independently).
 - Supports incremental timing (useful in large SoCs).
- Flat netlist:**
 - All paths are analyzed in one design → longer runtime.
 - No module boundaries.
 - Can potentially give slightly better optimization, but not scalable for large designs.

In today's SoC designs, hierarchical STA is widely used for efficiency and scalability.

6 How Does Hierarchy Impact Debugging ?

- Hierarchical netlist makes it easy to trace signals and violations module by module.
- You can quickly locate the source of a timing, DRC or connectivity issue within a specific block.
- Flat netlist is harder to debug because all instances are in one level with no module names.
- Tools like Innovus, PrimeTime, etc. show clear hierarchy, making debugging more efficient.



With hierarchy, you can focus on the problematic block instead of searching the entire design.

7 How to Inspect Hierarchy in Innovus ?

Useful Commands

```
# Show top module
get_design_info
# List all instances at current level
get_db_insts
# Show hierarchical instances
get_db_insts -hier
# Show sub-instances of a block
get_db_insts u_cpu/*
# Show module (cell) name of an instance
get_db_insts u_cpu -p2 .master
```

Example Output (Simplified)

```
Innovus> get_design_info
Top design: top
Innovus> get_db_insts -hier | head
top/u_cpu
top/u_cpu/u_alu
top/u_cpu/u_ctrl
top/u_mem
top/u_mem/u_sram
top/u_periph
top/u_periph/u_uart
```

Other Useful Commands

- report_hierarchy (Shows hierarchy tree)
- report_cells -hier (List cells with hierarchy)
- get_db_modules (List all modules)
- get_db_insts <inst> -p2 (Get instance details)

8 Summary

Topic	Key Point
Hierarchical netlist	Keeps module hierarchy, easier to debug and manage.
Flat netlist	No hierarchy, single level, harder to debug.
Placement impact	Hierarchy enables block-wise floorplanning.
STA impact	Hierarchical STA is faster and scalable.
Debugging	Hierarchy helps in quick root cause analysis.
Innovus inspection	Use get_db, report_hierarchy and related commands.



Key Takeaway: Hierarchy is your friend! It improves readability, placement control, STA scalability and debugging.

"Good hierarchy makes a big design feel small."



Gate-Level v
Standard Cells

Standard Cells Inside Gate-Level v

Common Cells in a Netlist and What They Do

Same Logic
Different Drive
Different Trade-offs

1 What Cells Are Present in Netlist ?

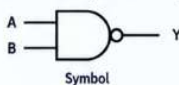
- Gate-level v (synthesized netlist) contains standard cells from a technology library.
- Cells include logic gates, flip-flops, multiplexers, clock gating cells, tie cells, etc.
- Cell names follow the format: <CELL>_<DRIVE>[_<VT>]

Example (snippet from netlist):

```
U1 NAND2_X1 ( .A(n1), .B(n2), .Y(n3) );
U2 DFF_X1 ( .D(n3), .CLK(c1k), .Q(n4) );
U3 ICG_X2 ( .CLK(c1k), .EN(en), .GCLK(g1k) );
U4 TIEHI_X1 ( .Y(vdd_net) );
```

2 NAND Cell

- NAND = NOT(AND)
- Most commonly used gate.
- Available in multiple drive strengths.



Symbol

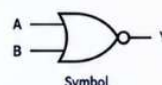
Example Cell Names

NAND2_X1 NAND2_X2 NAND2_X4
(2-input NAND)

Same logic, different drive strength (different area / power / timing).

3 NOR Cell

- NOR = NOT(OR)
- Used in many logic functions.



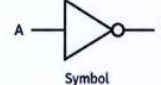
Symbol

Example Cell Names

NOR2_X1 NOR2_X2 NOR2_X4
(2-input NOR)

4 INV Cell

- INV = Inverter (NOT)
- Used for signal inversion.



Symbol

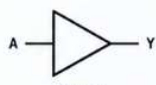
Example Cell Names

INV_X1 INV_X2 INV_X4

Often used for buffering or fixing inverted logic.

5 BUF Cell

- BUF = Buffer (no inversion)
- Used for driving stronger load.



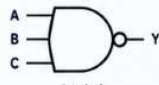
Symbol

Example Cell Names

BUF_X1 BUF_X2 BUF_X4

6 AOI Cell

- AOI = AND-OR-Invert
- Common complex gate.



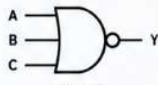
Symbol

Example Cell Names

AOI21_X1 AOI22_X1
(e.g., 2 AND inputs, 1 OR input)

7 OAI Cell

- OAI = OR-AND-Invert
- Common complex gate.



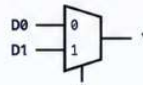
Symbol

Example Cell Names

AOI21_X1 AOI22_X1
(e.g., 2 OR inputs, 1 AND input)

8 MUX Cell

- MUX = Multiplexer
- Selects one of the inputs.



Symbol

Example Cell Names

MUX2_X1 MUX4_X1

9 DFF Cell

- DFF = D Flip-Flop
- Stores 1 bit of data (edge triggered).



Symbol

Example Cell Names

DFF_X1 DFF_X2 DFF_X4

10 SDF Cell

11 ICG Cell

12 Tie Cells

13 Different Drive Strengths

14 Different Vt Cells

- Sdff = Scan D Flip-Flop
- Used in DFT (scan).

Symb.El (Symbol (conceptual))

Example Cell Names

Sdff_X1 Sdff_X2

Has scan enable and scan input.

- ICG = Integrated Clock Gating
- Gates the clock based on enable.

Symbol (conceptual)

Example Cell Names

ICG_X1 ICG_X2

Used for power saving.

- Tie cells provide constant logic 0 or 1.
- Used to tie unused inputs.

Example Cell Names

TIEHI_X1 TIELO_X1

Connects unused inputs to a known value.

- Same function, different drive strength.
- Higher drive → larger area & power, better timing (can drive larger load).

Example: NAND2 cells

NAND2_X1 → lowest drive
NAND2_X2 → 2x drive
NAND2_X4 → 4x drive
NAND2_X8 → higher drive

Same logic, different area, power and timing.

- Vt = Threshold voltage
- Available in Low-Vt (L), Regular (R), High-Vt (H) variants.
- Low-Vt → faster (more leakage)
- High-Vt → slower (less leakage)
- Regular → balance

Example cell names

NAND2_X1_L (Low Vt)
NAND2_X1_R (Regular Vt)
NAND2_X1_H (High Vt)

Used to meet performance and leakage (power) requirements.

Key Takeaway: A gate-level netlist is built using standard library cells. Same logic can have different drive strengths and Vt options, which directly impact area, power and timing. Understanding these cells helps in timing, power, and physical design.

"Same function, different cells, different trade-offs."

Instance Name vs Reference Cell Name

Different Roles | Same Meaningful Connection | RTL → Netlist → LIB/LEF → Silicon

Right Names Build the Right Chips!

1 What is Instance Name ?

- Instance name is the unique name of a cell instance in the netlist.
- Each instance represents one physical copy of a cell in the design.
- You can have multiple instances of the same reference cell, but each instance name must be unique.

Example (from gate-level .v)

```
NAND2_X1 U1 ( .A(a), .B(b), .Y(n1) );
NAND2_X1 U2 ( .A(c), .B(d), .Y(n2) );
INV_X2 U3 ( .A(n1), .Y(out) );
```

Here, U1, U2, U3 are instance names. They are user/tool generated unique names.

2 What is Reference Cell Name ?

- Reference cell name is the name of the cell defined in the technology library (.lib).
- It specifies the type, function and drive strength of the cell.
- Many instances can use the same reference cell.

Example (from gate-level .v)

```
NAND2_X1 U1 ( .A(a), .B(b), .Y(n1) );
NAND2_X1 U2 ( .A(c), .B(d), .Y(n2) );
INV_X2 U3 ( .A(n1), .Y(out) );
```

Here, NAND2_X1, INV_X2 are reference cell names (taken from the .lib).

3 Key Differences

Aspect	Instance Name	Reference Cell Name
Meaning	Unique name of a cell in the netlist	Name of the cell defined in .lib
Example	U1, U2, U3, inst_1	NAND2_X1, INV_X2
Uniqueness	Must be unique in the design	Can be used multiple times
Represents	A specific physical copy of a cell	Cell type / function / drive strength
Comes from	Generated by synthesis tool (can be user-defined)	Defined in technology library (.lib)
Physical existence	Yes, as an individual placed cell	No, it is a template (master cell) used to create instances

4 Where Does Instance Exist Physically ?

- Each instance becomes a placed cell on the chip after placement (Innovus).
- It has a definite location (X, Y) and orientation.
- You can see instances in the layout (DEF/Innovus GUI).

Placed instances in layout (top view)

Each instance (U1, U2, U3) is a physical cell placed on silicon.

5 Where Does Reference Cell Come From ?

- Reference cell names come from the technology library (.lib).
- The .lib file contains timing, power, area and functional information for each cell (e.g., NAND2_X1, INV_X2).
- These cells are provided by the foundry or standard cell library vendor (e.g., TSMC, Samsung, Intel).

Example (.lib excerpt)

```
cell (NAND2_X1) {
  area : 1.25;
  pin (A) { direction : input; }
  pin (B) { direction : input; }
  pin (Y) { direction : output; }
  timing() { ... }
}
```

Library defines reference cells and their characteristics.

6 How .lib and LEF Connect with .v ?

Gate-level .v (Netlist)

```
NAND2_X1 U1 ( ... );
```

.lib (Timing, Power, Area)

- Cell name: NAND2_X1
- Timing arcs
- Power, area, constraints

LEF (Physical Layout)

- Cell name: NAND2_X1
- Cell size (width, height)
- Pin locations
- Physical shape (obstructions, layers)

Same cell name across all files.

7 Overall View in Design Flow

Uses reference cell names and instance names → Provides electrical characteristics → Provides physical geometry → Places each instance on silicon using LEF → GDSII (Final Layout) (Instances are now physical cells on chip)

Example: Putting It All Together

.v (Netlist)

```
NAND2_X1 U1 ( .A(a), .B(b), .Y(n1) );
```

.lib (timing/power)

```
cell (NAND2_X1) { ... }
```

LEF (physical)

```
MACRO NAND2_X1
  SIZE 1.4 BY 0.8 ;
  PIN A ...
  PIN B ...
  PIN Y ...
```

Same reference cell name (NAND2_X1) is used in .v, .lib and LEF, and each instance (U1) is placed physically on the chip.

Key Takeaway:

- Instance name = unique placed cell in the design.
- Reference cell name = cell type from library (.lib).
- .lib connects functionality → timing/power, LEF connects functionality → physical layout.
- Same cell name across .v, .lib and LEF ensures a consistent and correct design flow.

"Same cell name. Different views. One complete chip."

First Tool Commands After Receiving RTL (.v)

Understand, Elaborate, Link and Check Before Synthesis

Right Commands for a Clean Start Better Results!

1 Cadence Genus – Basic Flow (Priority)

- Genus automatically analyzes, elaborates and links the design with simple commands.
- Typical first commands after receiving RTL:

Genus Commands

```
read_hdl {rtl_files/*.v} # Read RTL files
elaborate # Elaborate design
check_design # Check for errors/warnings
report_design # Design summary
```

These commands help you quickly verify that the RTL is complete and ready for synthesis.

2 Synopsys DC – Basic Flow

- In Design Compiler, you need to explicitly analyze, elaborate and link the design.
- Typical first commands after receiving RTL:

DC Commands

```
analyze -format verilog {rtl_files/*.v}
elaborate top_module
current_design top_module
link
check_design
check_timing
```

Make sure to set the correct top module name before link and check commands.

3 Overall Flow

Initial RTL Checks (Read / Analyze, Elaborate, Link)

Before Synthesis (Set current design, Check design)

4 What is analyze ?

- Reads and parses the RTL source files

5 What is elaborate ?

- Builds the design hierarchy from the

6 What is link ?

- Resolves all module references in the

7 What is check_design ?

- Performs consistency checks on the

https://onedrive.live.com/personal/938043310494b6ef/_layouts/15/Doc.aspx?sourcedoc={0494b6ef-4331-2080-8093-7a0000000000}&action=edit&w...

9/14

(e.g., Verilog) and stores the design information in the tool's memory.

- Checks for syntax errors.
- Does not build the full design hierarchy (that happens during elaboration).

Example (DC)
analyze -format verilog (rtl_files/*.v)

In Genus, read_hdl performs a similar function as analyze.

analyzed RTL.

- Creates an internal representation of the design (modules, instances, ports, parameters).
- Identifies the top module (you can specify it, or the tool can infer it).

Example (DC)
elaborate top_module

In Genus, elaborate performs the same function.

design hierarchy.

- Looks for the definitions of instantiated modules.
- Uses the analyzed and elaborated information to connect everything together.
- If a module definition is missing, it will be reported as an unresolved reference or left as a black box.

Example (DC)
link

design after linking.

- Reports common issues such as:
 - Unresolved references
 - Multiply driven nets
 - Undriven nets
 - Combinational loops
 - Latch inference (warnings)
 - Other structural problems

Example (DC)
check_design

Always fix all errors (and critical warnings) before proceeding to synthesis.

8 What is current_design ?

- Sets the current working design (usually the top module).
- All subsequent commands (e.g., link, check_design, constraints) will be applied to this design.
- You can change it to work on a submodule if needed.

Example (DC)
current_design top_module

9 What is Unresolved Reference ?

- Occurs when the tool cannot find the definition of an instantiated module.
- It may be due to:
 - Missing RTL file
 - Incorrect file list
 - Wrong module name
 - Module not analyzed

Example Error Message (DC)
Warning: Unresolved reference to 'uart_rx' in module 'top' (UID-4)

Fix the file list or provide the missing module definition.

10 What is Black Box ?

- A module instance for which the tool does not have the RTL definition.
- Treated as a black box (no internal logic, only ports are known).
- Used intentionally for IPs (e.g., memory macros), or created unintentionally due to missing files.

Inputs → **Black Box** → Outputs

Black boxes are allowed for hard IPs, but unintended black boxes should be fixed.

11 What is unify ?

- Creates unique copies of modules that are instantiated multiple times.
- Useful when you want to apply different constraints or optimizations to each instance.
- After unify, each instance gets its own separate module definition.

Example (DC)
unify
or unify <module_name>

Without unify, multiple instances share the same module definition.

Key Takeaway:

- Use the basic commands (analyze/elaborate/link/check_design) to make sure the RTL is complete and error-free.
- Fix unresolved references and unintended black boxes before synthesis.
- Use unify only when you need separate copies of modules.

"Clean RTL, Clean Setup, Smooth Synthesis."

Gate-Level to Layout with Innovus

First Tool Commands After Receiving Gate-Level .v

Set Up the Design Environment, Read Files, Initialize and Check

Right Setup for a Successful Physical Design!

1 Innovus Flow - First Commands (Gate-Level .v)

Typical Command Sequence

```
# 1. Read technology file
set init_lef_file "tech.lef"
# or: read_physical -technology tech.lef

# 2. Read libraries (LEF for physical, .lib for timing)
set lef_files {list stocell.lef iio.lef}
read_lef $lef_files
set lib_files {list slow.lib typical.lib fast.lib}
set_db library $lib_files

# 3. Read gate-level netlist
read_netlist top.v

# 4. Read MMMC (optional but recommended)
read_mmmc mmmc.tcl

# 5. Initialize design
init_design

# 6. Check design
check_design
check_timing
```

2 How Does Innovus Read a Netlist ?

- Innovus reads the gate-level netlist using the read_netlist command.
- It parses the Verilog file, creates the design hierarchy and matches each instance with a cell definition from the loaded libraries (LEF/.lib).
- Only the structural (gate-level) information is used, not behavioral code.
- All cells must exist in the loaded libraries, otherwise they appear as unresolved (black boxes).

Example
read_netlist top.v

3 What is init_design ?

- init_design creates the in-memory design database in Innovus using the loaded technology, libraries, netlist and MMMC.
- It performs design setup, links the design, builds internal data structures and prepares for floorplanning.
- You can specify the top module: init_design -top <top>

Example
init_design -top top

What Checks Happen During Initialization ?

- Checks for undefined / unresolved cells.
- Verifies that all referenced libraries are loaded.
- Builds the design hierarchy.
- Reads constraints (if MMMC is loaded).
- Checks for missing modules or files.
- Reports basic design statistics (cells, nets, hierarchy).
- Issues warnings/errors for any inconsistencies.

4 How to Detect Unresolved Cells ?

- Unresolved cells appear when Innovus cannot find the cell definition in the loaded libraries.
- Use the following commands:

```
report_unresolved # List unresolved cells
report_black_boxes # List black box cells
check_design -unresolved # Detailed check
```

Example Output (simplified)

```
# Unresolved Cell Instances
U1 NAND2_X1
U2 AOI21_X1
U3 XYZ_CELL (not found in libraries)
```

This cell is not present in the loaded .lib / LEF

5 How to Detect Missing Modules ?

- Missing modules occur when a module definition is not found in the netlist or in the loaded libraries.
- Use:

```
check_design -missing_modules # Check missing modules
report_modules # List all modules
check_design -hierarchy # View hierarchy
```

Example Output (simplified)

```
# Missing Module Definitions
Module 'uart_rx' is not defined
Module 'i2c_master' is not defined
```

Ensure all RTL/netlist files are read and all required libraries are provided.

6 How to Check Loaded Design ?

- Use the following commands to verify what is loaded in Innovus:

```
report_design # Design summary
get_db top # Show top module
get_db modules # List all modules
get_db insts # List instances
get_db cells # List cell types
report_hierarchy # Hierarchy view
```

Example Output (simplified)

```
Innovus> report_design
Top Module : top
#Modules : 12
#Instances : 13432
#Nets : 18450
#Cells : 15220
#Unresolved Cells : 0
```

If "Unresolved Cells" is 0, the design is properly loaded.

7 Useful Additional Checks

- Check for black boxes: report_black_boxes
- Check for library cells: get_db lib_cells
- Check timing libraries: get_db timing_libs
- Check MMMC scenarios: report_scenarios
- Check constraints: report_constraints

8 Key Points

- ✓ Read correct technology, LEF, timing libraries and MMMC before netlist.
- ✓ Make sure all cells are resolved (no black boxes).
- ✓ Verify the top module is correct.
- ✓ Check for missing modules and libraries.
- ✓ Use report commands to confirm the design is loaded properly.
- ✓ Fix all errors/warnings before proceeding to floorplan.

9 Summary Flow

1. Read technology (LEF)
2. Read libraries (LEF + .lib)
3. Read netlist (read_netlist)
4. Read MMMC
5. init_design
6. check_design (resolve all issues)
7. Verify with report commands

Key Takeaway: A clean and fully resolved design setup in Innovus is the foundation for a successful physical design.

"Read it right, resolve it all, and build a solid foundation!"

From RTL Netlist To Silicon

Complete .v File Checks Before PnR

Ensure a Clean, Error-Free Netlist for a Successful Physical Design

Check Early Save Time Avoid Re-spins!

1 Structure Checks Verify design hierarchy and module connections

Check Item	What to Check	Why Important	Common Commands (Innovus)
Top module	- Correct top module name - All ports present	Design is loaded correctly	init_design -top <top> get_db design_name
Module hierarchy	- All sub-modules present - Hierarchy is complete	No missing modules	report_hierarchy get_db modules
Port mistakes	Port count, names, direction match between instances	Avoid connection errors	check_design

2 Connectivity Checks Ensure correct signal connectivity

Check Item	What to Check	Why Important	Common Commands (Innovus)
Floating nets	Nets with no driver and no load	May indicate missing connections	report_floating_nets check_design
Undriven nets	Nets with load but no driver	Will remain X/unknown in silicon	report_undriven_nets check_design
Unloaded	Output ports/nets	May have design issues	report_unloaded_nets

Check Item	What to Check	Why Important	Common Commands (Innovus)
Unresolved references	No undefined modules/cells	Missing definitions will cause errors	report_unresolved check_design -report
Black boxes	Identify and review any black boxes	May cause issues in placement/STA	report_black_boxes get_db_black_boxes

Example: Unresolved Reference

```
Innovus> report_unresolved
Module 'uart_rx' is not defined
Module 'i2c_master' is not defined
```

All modules should be resolved with no unintended black boxes (except for intentional hard macros).

Example: Multi-Driven Net

```
Innovus> report_multi_driver_nets
Net: ctrl_signal
Driver1: U1/Z (AND2_X1)
Driver2: U2/Z (OR2_X1)
```

All nets should have exactly one driver (unless it is a valid bus or tri-state structure).

3 Cell Checks

Verify standard cells and special cells

Check Item	What to Check	Why Important	Common Commands (Innovus)
Unknown cells	Cells not found in loaded libraries (LEF/lib)	Cannot be placed or routed	report_unknown_cells report_unresolved
dont_use cells	Check for cells marked as dont_use	May cause placement or signoff issues	report_dont_use_cells report_cells -dont_use
Unexpected latches	Look for latch cells (e.g., LAT, LATCH)	May indicate RTL issues	report_cells -hier -filter {ref_name == *LAT*}
Tie cells	Check tie high/low cells (TIEH, TIELO)	Used for constant logic (should exist in libs)	report_cells -hier -filter {ref_name == *TIE*}
Buffers/inverters	Check presence and usage (BUF, INV)	Common for buffering and fixes	report_cells -hier -filter {ref_name == *BUF* ref_name == *INV*}

Example: Unknown Cell

```
Innovus> report_unknown_cells
U123 ( XXXX_X1 )
U456 ( SOME_CELL )
```

All cells in the netlist must exist in the loaded libraries. Investigate any unknown or dont_use cells.

Tie cells, buffers and inverters are normal and expected.

4 Clock Checks

Verify clock structure and related logic

Check Item	What to Check	Why Important	Common Commands (Innovus)
Clock ports	- Identify all clock input ports - Check correct constraints (MMC)	Required for timing analysis and CTS	report_ports -clock report_clocks
Generated clocks	Check for generated/divided clocks	Ensure proper constraints and clock tree handling	report_clocks -generated report_clock_trees
Clock gating	Identify ICG cells (e.g., CLKGATEX)	Special handling during CTS and timing	report_cells -hier -filter {ref_name == *ICG*} report_clock_gating
Data cells in clock path	Ensure no combinational data cells in clock path (except intentional dividers)	Can cause glitches and timing issues	report_timing -from [get_clocks] -to [all_registers] -through (*) -worst 10

Example: Clock Gating Cell

```
Innovus> report_cells -hier -filter {ref_name == *ICG*}
U100 ICG_X1
U200 CLKGATE_X2
```

All clocks should be identified, constrained and free from unintended logic in the clock path.

5 Final Sign-off Checklist (Before PnR)

- No unresolved references or unknown cells
- No floating, undriven, unloaded or multi-driven nets
- Hierarchy and ports are correct
- Clocks properly identified and constrained
- No unwanted latches
- Only allowed/valid black boxes
- All required libraries (LEF, lib) are loaded

"A clean netlist today means a smooth PnR tomorrow."

Complete .v File Checks Before PnR

Catch Issues Early | Ensure a Clean, Constrained and DFT-Ready Netlist

Clean Netlist Smooth PnR Better Silicon!

1 Structure Checks

Verify design hierarchy and module definitions

Check Item	What to Check	Key Points
Top module	- Correct top module name - All ports present	Should match spec.
Module hierarchy	- All sub-modules present - Hierarchy is complete	No missing modules
Port matching	- Port count, names, direction - Match between instances and module definitions	Avoid connection errors
Unresolved references	No undefined modules/cells	Causes elaboration/linking errors
Black boxes	Identify and review any black boxes	Allowed only if intentional (e.g., hard IP)

Example: Check hierarchy (Innovus)

```
report_hierarchy
report_unresolved
```

Design hierarchy should be complete with no unresolved references.

2 Connectivity Checks

Ensure correct signal connections

Check Item	What to Check	Key Points
Floating nets	Nets with no driver and no load	May indicate missing connections
Undriven nets	Nets with load but no driver	Will remain X/unknown in silicon
Unloaded outputs	Output ports/nets with no load	May be a design issue or unused logic
Multi-driven nets	Nets with multiple drivers	Can cause functional and DRC issues

Example: Find connectivity issues

```
report_floating_nets
report_undriven_nets
report_unloaded_nets
report_multi_driver_nets
```

A net should have exactly one driver (unless it is a valid bus or tri-state).

3 Cell Checks

Verify standard cells and special cells

Check Item	What to Check	Key Points
Unknown cells	Cells not found in loaded libraries	Check lib is loaded and cell exists
dont_use cells	No cells marked as dont_use	May cause placement/signoff issues
Unexpected latches	Look for inferred latches (e.g., incomplete if/else)	Can cause functional and timing issues
Tie cells	Check tie high/low cells (TIEH, TIELO)	Used for constant logic
Buffers/inverters	Check presence and reasonable usage	Common for buffering and fixes

Example: Check unknown/dont_use cells

```
report_unknown_cells
report_cells -dont_use
```

All cells must exist in loaded libraries and meet design rules.

4 Clock Checks

Verify clock definition and structure

Check Item	What to Check	Key Points
Clock ports	- Identify all clock input ports - Check constraints (MMC)	Required for CTS and timing analysis
Generated clocks	Check for generated/divided clocks	Ensure proper constraints
Clock gating	Identify ICG cells (e.g., CLKGATE)	Handle correctly during CTS and timing
Data cells in clock path	Ensure no combinational data cells in clock path	Can cause glitches and timing issues

Example: Check clocks

```
report_clocks
report_clock_trees
report_timing -from [get_clocks *]
```

5 Reset Checks

Verify reset structure and connectivity

Check Item	What to Check	Key Points
Reset type	Identify reset type (synchronous/asynchronous, active high/low)	Check design spec and consistency
High fanout	Reset nets with very high fanout	May need buffering/optimization
Connectivity	Ensure reset is connected to all sequential elements	No floating/unconnected resets

Example: Check resets

```
report_reset
report_fanout -nets [get_nets rst*]
report_undriven_nets -filter {name == *rst*}
```

All sequential elements must be properly reset. Check fanout and use reset tree/buffers if required.

6 DFT Checks

Verify scan insertion and connectivity

Check Item	What to Check	Key Points
Scan ports	Check presence of scan ports (Scan_in, Scan_out, Scan_en, Scan_clk)	Must be present and connected
Scan FFs	Check number of scan FFs vs. expected	Ensure correct insertion
Scan chains	Verify chain connectivity, no breaks	Should form complete chains
Scan enable	Check scan enable signal and logic	Should be controllable during test mode

Example: DFT related checks

```
report_scan_chain
report_dft
report_port -filter {name == *scan*}
```

7 Timing Checks

Verify timing constraints and analysis

Check Item	What to Check	Key Points
All endpoints constrained	Check that all clocks, I/O and exceptions are defined	Use reports to find unconstrained endpoints
No unconstrained paths	Ensure there are no unconstrained paths	Unconstrained paths can cause timing failures later
Setup	Verify setup timing	No setup violations (WNS ≥ 0)
Hold	Verify hold timing	No hold violations (WHS ≥ 0)
DRVs	Check for design rule violations affecting timing (max transition, max capacitance, etc.)	Clean DRVs before PnR

Example: Timing reports

```
report_timing_summary
report_constraint -all_violators
report_timing_unconstrained
report_design_rules
```

All paths should be constrained and meet setup/hold timing with no DRVs.

8 Summary Checklist

- Top module and hierarchy are correct
- All ports match and no unresolved references
- No floating, undriven, unloaded or multi-driven nets
- All cells are known and valid (no dont_use)
- No unintended latches
- Clocks are properly defined and constrained
- Resets are correctly connected with acceptable fanout
- DFT insertion is complete and scan chains are connected
- All endpoints are constrained (no unconstrained paths)
- No setup/hold violations and timing DRVs are clean

9 Useful Innovus Commands

Purpose	Command
Design summary	report_design
Hierarchy	report_hierarchy
Unresolved references	report_unresolved
Connectivity	report_floating_nets report_undriven_nets
Cells	report_unknown_cells report_cells -dont_use
Clocks	report_clocks
Resets	report_reset
DFT	report_scan_chain
Timing	report_timing_summary
DRVs	report_design_rules
Ports	report_ports

Key Takeaway: A clean, well-verified .v file today saves time, avoids iterations, and leads to a smooth PnR and successful silicon.

"Check Early, Build Confidently!"

Common Netlist Issues in RTL-to-GDS Flow

Understand | Detect | Debug | Prevent

Clean Netlist Smooth PnR Reliable Silicon!

1 Floating Pins

Pins of a cell instance left unconnected

- A pin (input or output) of an instantiated cell is not connected to any net.
- Common for optional pins (e.g., scan, test, tie, power-down).
- May cause unknown behavior, warnings, or X-propagation in simulation.

Example

How to detect

- check_design
- report_floating_pins
- report_pins -unconnected

How to fix

- Connect to a valid net or tie cell (TIEHI/TIELO) if intentional.

2 Undriven Nets

Nets with no driver

- A net exists in the design but no cell or port drives it.
- The net value is unknown (X) in simulation.
- Usually happens due to missing connection, incomplete RTL, or removed logic during synthesis.

Example

How to detect

- report_undriven_nets
- check_design

How to fix

- Connect to a valid source or remove the unused net.

3 Unloaded Outputs

Output pins or nets with no load

- A net or output port is driven, but nothing is connected to it.
- May be intentional (test points, spare logic) or a design issue.
- Can lead to optimization removal or useless logic.

Example

How to detect

- report_unloaded_nets
- report_ports -unconnected

How to fix

- Connect to a valid load or mark as intentionally unused (e.g., dont_touch, keep).

4 TIEHI and TIELO

Constant logic 1 and 0 cells

- Tie cells are special standard cells that provide constant logic 1 (TIEHI) or logic 0 (TIELO) to drive unused inputs.
- Used to avoid floating pins and to tie-off signals like scan enable, mode selects, etc.
- These are real physical cells available in the standard cell library (.lib/.lef).

Symbols

Usage Example

```
U_TIE1 TIEHI_X1 (.Y (scan_en));
U_TIE0 TIELO_X1 (.Y (test_mode));
```

Use tie cells for all intentional constant connections. Do not leave pins floating.

5 Multi-Driven Nets

Nets with multiple drivers

- A net is driven by more than one source (two or more outputs).
- Creates logic conflict (0 vs 1) → undefined value (X).
- Usually a design bug (except for intentional tri-state or wired-OR structures, which are rare in modern ASIC flows).

Example

How to detect

- report_multi_driver_nets
- check_design

How to fix

- Ensure only one driver per net. Use proper muxing or arbitration logic if multiple sources exist.

6 Why Multi-Driven Nets are Dangerous ?

- Causes contention → undefined logic (X).
- Can lead to high current and potential cell damage in real silicon.
- Breaks synthesis optimizations and timing analysis.
- Difficult to debug if it reaches PnR (may cause DRC/LVS failures).

If it reaches PnR ?

- May fail during placement with multiple drivers on the same net.
- Can create short-circuit paths and unrealistic timing.
- Leads to DRC and LVS errors.
- May cause functional failure in silicon.

Debug Steps

- Run report_multi_driver_nets
- Trace the net in the tool (e.g., get_nets, all_fanin/all_fanout)
- Identify all drivers and fix the RTL or constraints
- Re-run synthesis and verify

7 Black Box

Module with no internal definition

- A black box is a module or cell for which the tool has only the interface (ports) but no internal implementation (no RTL or netlist).
- Treated as an unknown module during synthesis, STA or PnR.
- Used intentionally for hard macros, memories or third-party IPs.
- It can also happen unintentionally due to missing RTL files or wrong file order.

Example

```
module my_ip (input clk, input rst, output data);
// No implementation
endmodule
```

Why does it happen ?

- Missing RTL file
- Wrong file order
- Excluded from file list
- Encrypted RTL
- 3rd party IP (no RTL)
- Memory macro (pre-built)
- Intentional black box

8 Memory and IP Black Box

Pre-built macros or third-party blocks

Type	Description	Example	Why Black Box ?
Memory black box	Pre-built memory macros (SRAM, ROM, etc.)	sram_1rw, rom	Provided as a compiled netlist (.v and .lib/.lef, no RTL)
IP black box	Third-party or in-house hard IPs (PCIe, USB, DDR, CPU, etc.)	pcie_ip, ddr_ctrl	Often delivered as encrypted RTL or pre-compiled netlist

9 How to Detect and Handle Black Boxes ?

Detect

- report_black_boxes
- report_PnRn
- check_design
- report_hierarchy

Debug / Fix

- Verify all RTL files are read
- Check file order and paths
- Ensure correct top module is set
- For intentional IP/memories: provide .lib, .lef and netlist (or use synthesis/PD specific models)
- Use set_dont_touch or black box models if needed

Key Takeaways

- No floating pins, undriven nets, unloaded outputs or multi-driven nets.
- Use TIEHI/TIELO for constant connections.
- Resolve all black boxes with proper RTL/netlist and libraries.
- Fix all issues before PnR to ensure a smooth and reliable physical design.

"A clean netlist today builds a better silicon tomorrow."

Key Concepts for a Clean Netlist and Successful PnR

Good Netlist
Better PnR
Reliable Silicon !

High Fanout | Clock Gating | Scan Design | Netlist Verification | LEC

1 High Fanout

What it is, why it's a problem, and how it's fixed

What is High Fanout ?

- When a single net drives a large number of loads (fanout).
- Example: a control signal, reset, clock enable, or configuration signal driving hundreds or thousands of cells.
- There is no fixed number, but typically > 100-200 loads is considered high fanout (depends on technology and flow).

Why it's a Problem ?

- Large capacitive load → slower timing
- Higher power consumption
- Signal integrity issues (slew, noise)
- Harder to meet timing
- Can create placement congestion (many load cells spread across the chip)
- May violate max fanout or max transition constraints

How Synthesis Fixes It ?

- Inserts **buffers** (buffer tree)
- Replicates driver logic
- Uses dedicated high-fanout cells
- Optimizes via compile options (e.g., set_max_fanout)

What is Buffering ?

- Adding buffer cells (repeaters) to split the load into smaller groups.
- Creates a buffer tree (multiple stages) to drive all loads.
- Helps improve slew, reduce capacitance per stage, and meet timing.

2 Impact on Congestion

How high fanout affects placement and routing

- Loads of a high fanout net are spread across the chip.
- The buffer tree and routing resources (wires, tracks) can consume large area.
- Increases local and global routing congestion.
- May require larger keepout/spacing, leading to sub-optimal placement.

Key Takeaway

- High fanout nets need proper buffering and planning in synthesis and PnR.
- Early detection and fixing helps reduce congestion and improves timing, power, and routability.

3 Clock Gating and ICG Cell

Save power by gating the clock

What is Clock Gating ?

- Technique to stop the clock to a block when it is not active.
- Reduces dynamic power (no unnecessary switching).
- Implemented using a special latch-based clock gating cell (ICG).

ICG Cell (Integrated Clock Gating)

- A standard cell that safely gates the clock to avoid glitches.
- Contains a latch to synchronize the enable signal with the clock.
- Typical pins: CLK, EN, GCLK (gated clock).
- Ensures glitch-free clock gating.

Example

Benefits

- Lower dynamic power
- Reduces switching activity
- Widely used in ASIC designs

Notes

- Synthesis can automatically insert ICG cells (use clock_gating options).
- Ensure proper constraints (set_clock_gating_style).
- Verify with STA (gated clocks are handled correctly).

4 Scan Flip-Flop (Scan FF)

Make the design testable

What is Scan FF ?

- A flip-flop with an additional scan input (SI), scan output (SO), and scan enable (SE).
- Used for DFT (Design for Testability).
- Works in two modes:
 - Functional mode (SE = 0): normal data path.
 - Scan mode (SE = 1): forms a shift register (scan chain) for testing.

Scan FF Symbol

How it Works ?

SE	Mode	Function
0	Functional	Q <- D (normal operation)
1	Scan	Q <- SI (scan shift operation)

Why It's Used ?

- Enables fault testing (stuck-at faults, etc.).
- Improves test coverage.
- Required for production test (ATE).

5 Scan Enable (SE)

Control signal for scan mode

What is Scan Enable ?

- A global control signal that selects between functional and scan mode.
- When SE = 0 → design works normally.
- When SE = 1 → all scan FFs are connected in a chain and data is shifted in/out.
- Usually a top-level input (test mode control).
- Typically connected to a dedicated test pin and tied low during normal operation.

Example: Scan Chain

Notes

- Ensure SE is properly constrained in STA (set_case_analysis).
- During normal mode, SE should be tied low.

6 Netlist Verification and LEC

Ensure design is correct after synthesis

How to Verify a Netlist ?

- Check for no errors or unresolved references (check_design).
- Verify all cells exist in the libraries.
- Run structural checks (connectivity, floating nets, etc.).
- Ensure constraints are met (timing, DRC, etc.).
- Simulate gate-level netlist (optional).
- Use formal verification (LEC) to ensure functional equivalence with RTL.

What is LEC (Logical Equivalence Check) ?

- LEC verifies that the synthesized netlist is functionally equivalent to the original RTL.
- Compares logic (not timing) between RTL and netlist.
- Ensures no unintended logic changes during synthesis (optimizations, retiming, etc.).
- Common tools: Synopsys Formality, Cadence Conformal.

LEC Flow (Simplified)

(read_verilog) (read_verilog) (set_top, link) (verify) (0 mismatches)

✓

Final Takeaway:

- Handle high fanout, clock gating, scan design, and verify the netlist (including LEC) before PnR to ensure better timing, lower power, fewer issues, and a successful silicon.

"Verify Today,
Avoid Re-spins Tomorrow."

RTL to PnR
 Debug Early
 Save Time

Common Errors When Reading .v in EDA Tools

Understand the Error | Find the Root Cause | Apply the Fix | Build a Clean Design

Clean Input
 Smoother Flow
 Successful Silicon!

1 Cannot Find Module

Module definition is not found in the files

Example Error
(DC) Error: Cannot find module 'uart_rx' (Cadence) *E,ELABNOMOD: Module uart_rx is not defined.

Cause

- RTL file missing
- File not added in analyze/read_hdl
- Wrong file order
- Type in module name

Fix

- Check file list and paths
- Add all RTL files
- Ensure correct file order (sub-modules before top)
- Verify module name spelling

2 Unresolved Reference

Module/cell is referenced but not found

Example Error
(DC) Warning: Unresolved reference 'U_MEM' (Cadence) *W,UNRES: Unresolved reference U_MEM.

Cause

- Missing RTL or library
- Black box (intentional or accidental)
- Third-party IP not provided

Fix

- Provide the missing RTL or library
- If intentional, set as a black box
- Check file order and library paths

3 No Cells in Library

Tool cannot find any design cells

Example Error
(DC) Error: No cells in library '(typical. lib)' (Cadence) *E,NOCELL: No cells found in the library.

Cause

- Library not read/loaded
- Wrong library path
- Incompatible library format

Fix

- Read the correct .lib files
- Check library path and syntax
- Use valid PDK libraries (right corner/tech)

4 Port Mismatch

Instance ports do not match module definition

Example Error
(DC) Error: Port 'rst_n' not found in module 'uart_rx' (Cadence) *E,PORTMAP: Formal port 'rst_n' not found.

Cause

- Port name mismatch
- Different port count/order
- Using named mapping with wrong name

Fix

- Check module definition and instantiation
- Ensure port names and widths match
- Use positional mapping if needed

5 Multiple Drivers

A net is driven by more than one source

Example Error
(DC) Error: Net 'data' has multiple drivers. (Cadence) *E,MORV: Net data has multiple drivers.

Cause

- Two or more outputs driving the same net
- Tri-state/mux not used properly
- RTL coding issue

Fix

- Check the drivers using report_net / report_driver
- Use proper mux or tri-state
- Fix RTL to ensure single driver

6 Floating Nets

A net is not connected to any driver

Example Error
(DC) Warning: Net 'ctrl' is floating. (Cadence) *W,FLOAT: Net ctrl is floating (no driver).

Cause

- Missing connection
- Unconnected port
- Optional logic not tied (scan, test, etc.)

Fix

- Trace the net and connect to a driver
- Tie to TIEHI/TIELO if intentional
- Check unconnected module ports

7 Missing Clock

No clock defined for sequential logic

Example Error
(DC) Warning: No clock found. (Cadence) *W,NOCLK: No clock defined in the design.

Cause

- Clock not defined in SDC
- Wrong clock port name
- Clock generated internally but not constrained

Fix

- Define clocks in SDC (create_clock)
- Verify clock port names
- Check generated clocks (create_generated_clock)

8 Missing SDC

No timing constraints provided

Example Error
(DC) Warning: No constrain file specified. (Cadence) *W,NOCONSTRAINT: No timing constraints found.

Cause

- SDC file not provided
- Wrong SDC path
- Incomplete constraints (clocks, IO, etc.)

Fix

- Provide the correct SDC file
- Define clocks, IO delays, and exceptions
- Check SDC is read successfully

9 Black Box

Module treated as unknown (no internal definition)

Example Error
(DC) Warning: Module 'mem_top' is a black box. (Cadence) *W,BLKBKX: Module mem_top is defined as black box.

Cause

- RTL not provided
- Intentional black box (e.g., IP, memory)
- Encrypted or pre-compiled model

Fix

- Provide RTL if needed
- Keep as black box for hard IP/memory
- Use set_dont_touch or set_black_box if intentional

10 Missing Library Cell

Referenced cell not found in technology library

Example Error
(DC) Error: Cell 'NAND2_X1' not found in library. (Cadence) *E,CELLNF: Cannot find cell NAND2_X1.

Cause

- Wrong/missing .lib file
- Cell not available in the selected library
- Mismatch between netlist and library (e.g., wrong PDK/corner)

Fix

- Read the correct standard cell libraries
- Check library versions and corners
- Verify cell name (NAND2_X1, nand2_x1, etc.)

11 Wrong Top Module

Specified top module is not correct

Example Error
(DC) Error: Current design 'top' not found. (Cadence) *E, TOP: Top module 'top' is not defined.

Cause

- Wrong top module name
- Top module not included in file list
- Multiple possible top modules

Fix

- Specify correct top module (elaborate top or init_design -top)
- Check module names using report_design or get_designs
- Ensure top module is in the RTL files

12 Netlist / LEF Mismatch

Mismatch between netlist cells and LEF

Example Error
(Innovus) *E,LEF-003: Cannot find cell NAND2_X1 in loaded LEF. (General) Cell not found in technology file (LEF).

Cause

- LEF for that cell not loaded
- Wrong PDK version
- Netlist from different technology
- Cell name mismatch (NAND2_X1 vs NAND2X1)

Fix

- Load correct LEF files
- Use matching PDK (same version as .lib)
- Check cell names and naming conventions
- Ensure tech LEF, macro LEF and stdcell LEF are all loaded

✓

Key Takeaways

- Read all required files (RTL, .lib, LEF, SDC) in correct order.
- Resolve all errors and warnings before synthesis/PnR.
- A clean .v and constraint setup saves time and avoids iterations.

"Fix the input, not the output.
A clean netlist today, a smoother tapeout tomorrow."

Synthesis to PnR
 Reports Tell the Story
 of Your .v File

Reports Related to .v File Handoff

Complete Synthesis Package for a Successful PnR

Right Reports
 Right Confidence
 Better Silicon!

1 What Reports Accompany Synthesized .v ?

A typical synthesis handoff package includes:

File/Report	Purpose
Synthesized netlist (.v)	Gate-level netlist for PnR
SDC (.sdc)	Timing constraints used
Timing report	Setup/hold timing results
Area report	Cell count and area
Power report	Power estimation
QoR report	Overall quality summary
DRC/DRV report	Design rule checks
Cell distribution report	Standard cell usage
Clock gating report	Inserted ICG cells and savings
Scan chain report (DFT)	Scan chain details
Removed registers report	Registers removed/optimized
Synthesis log	Complete command log and messages

2 Timing Report

Shows whether design meets timing constraints.

Timing Path Group: clk
 Startpoint: u1/reg1 (posedge clk)
 Endpoint: u2/reg2 (posedge clk)

Delay Type	Arrival	Required	Slack
max	1.23	1.50	0.27 (MET)
min	0.12	0.00	0.12 (MET)

Key Information

- Setup and hold timing
- Worst Negative Slack (WNS) / Total Negative Slack (TNS)
- Path details (startpoint, endpoint, delay)
- Timing for all clock domains and modes

Goal:
 WNS ≥ 0
 TNS = 0

3 Area Report

Shows the area and cell count of the design.

Design: top

Instance	Cells	Area (um^2)	%
Sequential	12567	52345.67	52.3
Combinational	18734	47765.21	47.7
Total	31301	100110.88	100.0

Key Information

- Total area and cell count
- Combinational vs sequential area
- Hierarchical area breakdown
- Area per module/block

Goal:
 Meet area target
 (e.g., from spec)

4 Power Report

Shows estimated power consumption.

Power Report (mW)

Internal Power	: 12.34
Switching Power	: 8.76
Leakage Power	: 3.21
Total Power	: 24.31

Key Information

- Internal, switching, leakage power
- Total power (per module/corner)
- Power per module or block
- Power by clock domain

Goal:
 Meet power budget
 (e.g., from spec)

5 QoR Report

Overall quality of results.

QoR Summary

WNS (ns)	: 0.12
TNS (ns)	: 0.00
Area (um^2)	: 100110.88
Total Power (mW)	: 24.31
DRC Violations	: 0
Cell Count	: 31301

Key Information

- Summary of timing, area, power, DRV
- Used to compare different synthesis runs
- Helps decide if QoR meets project goals

Goal:
 Meet all targets
 (timing, area, power, no DRVs)

6 DRV Report

Checks for design rule violations.

DRV Summary

Max transition violation	: 0
Max capacitance violation	: 0
Max fanout violation	: 0
Max cell load violation	: 0
Max area violation	: 0

Key Information

- Max transition, capacitance
- Fanout violations
- Cell load violations
- No DRVs should remain

Goal:
 0 DRVs
 before handoff

7 Cell Distribution Report

Shows usage of different standard cells.

Cell Type	Count	%
INV	2345	7.5
NAND2	5678	18.1
NOR2	3456	11.0
OFF	8901	28.4
ICG	123	0.4
TIEHI/TIELO	45	0.1
BUF	2345	7.5
...	...	...

Key Information

- Count of each cell type
- Sequential vs combinational cells
- Special cells (ICG, TIE, scan, etc.)
- Helps in library and PnR planning

Goal:
 Expected cell mix
 (no unwanted cells)

8 Clock Gating Report

Shows inserted clock gating (ICG) cells and savings.

Clock Gating Report

9 Scan Chain Report

Shows DFT insertion results.

Scan Chain Report

Total scan chains : 10

10 Removed Registers Report

Shows registers removed or optimized.

Removed Registers Report

11 What Goes in Handoff to PnR ?

Complete synthesis deliverables.

Deliverable	Description
Netlist (.v)	Gate-level synthesized netlist
SDC (.sdc)	Timing constraints used

Total clocks : 8

Gated clocks : 6

ICG cells inserted : 123

Estimated power savings : 15.2 %

Key Information

- Number of clocks and gated clocks
- ICG cells inserted (e.g., CLKGATE_X1)
- Estimated power savings
- Check for missing gating opportunities

Goal:

Proper clock gating for low power (no functional issues)

Total scan FFs : 4521

Min chain length : 402

Max chain length : 468

Average length : 452

Scan enable port : scan_en

Scan in port : scan_in

Scan out port : scan_out

Key Information

- Number of scan chains and scan FFs
- Chain lengths (min/max/avg)
- Scan ports (scan_in, scan_out, scan_en)
- Ensure full scan coverage

Goal:

All FFs scanned with balanced chain lengths

Total registers : 12507

Removed registers : 234 (1.86%)

Reasons:

- Constant propagation : 120
- Redundant logic : 80
- Unused logic : 34

Key Information

- Number of registers removed
- Reason for removal (constant, redundant, unused)
- Ensure no unintended functional loss
- Cross-check with RTL/LEC

Goal:

Only safe removals (no functional loss)

Timing report

Setup/hold timing results

Area report

Area and cell count

Power report

Power estimation

QoR report

Overall summary

DRV report

Design rule checks

Cell distribution

Standard cell usage

Clock gating report

ICG details and savings

Scan chain report

DFT scan chain information

Removed registers

Registers removed/optimized

Synthesis log

Complete tool log and run script

Key Takeaway:

- A synthesized .v is not just a netlist — it comes with reports that prove the design meets timing, area, power, DRC/DRV and DFT requirements.
- Always review all reports before handoff to ensure a smooth PnR flow.

“Good Reports Today,
Fewer Surprises in PnR Tomorrow.”

https://onedrive.live.com/personal/938043310494b6ef/_layouts/15/Doc.aspx?sourcedoc={0494b6ef-4331-2080-8093-7a0000000000}&action=edit&...

14/14